



NEIL GOGTE INSTITUTE OF TECHNOLOGY

A unit of Keshav Memorial Technical Educational Society (KMTES)
(Approved by AICTE, New Delhi & Affiliated to Osmania University, Hyderabad)
D.No:10TC-111, Kachivanisingaram (V), Uppal, Hyderabad-500088, Telangana.



COMPUTER ORGANIZATION LAB [PC 451 CS]

LABORATORY MANUAL

For

B.E – IV Semester

Prepared By

Mrs V.Praveena

Assistant Professor

Name:

Roll. No:

Academic Year:



NEIL GOGTE INSTITUTE OF TECHNOLOGY

*A unit of Keshav Memorial Technical Educational Society (KMTES)
(Approved by AICTE, New Delhi & Affiliated to Osmania University, Hyderabad)
D.No:10TC-111, Kachivanisingaram (V), Uppal, Hyderabad-500088, Telangana.*

VISION

To be a leader in producing industry-ready and globally competent engineers to make India a world leader in software products and services.

MISSION

- To provide a conducive learning environment that inculcates problem solving skills, professional and ethical standards, lifelong learning through multi modal platforms and prepare students to become successful professionals.
- To forge industry-institute partnerships to expose students to the technology trends, work culture and ethics in the industry.
- To provide quality training to the students in the state-of-the-art software technologies and tools.
- To encourage research-based projects/activities in emerging areas of technology.
- To nurture entrepreneurial spirit among the students and faculty.
- To induce the spirit of patriotism among the students that enables them to understand India's challenge and strive to develop effective solutions.



NEIL GOGTE INSTITUTE OF TECHNOLOGY

*A unit of Keshav Memorial Technical Educational Society (KMTES)
(Approved by AICTE, New Delhi & Affiliated to Osmania University, Hyderabad)
D.No:10TC-111, Kachivanisingaram (V), Uppal, Hyderabad-500088, Telangana.*

DEPARTMENT VISION & MISSION

DEPARTMENT VISION

To be a leading department in teaching, research and placements among the Computer Science and Engineering departments in the region producing globally competent and socially responsible graduates in the most conducive academic environment.

DEPARTMENT MISSION

- To provide the state-of-the-art infrastructure to the faculty and students that facilitates continuous professional development and research in fundamental aspects and emerging computing trends alike.
- To impart the technological know-how that helps students to design technical solutions for social needs and kindle entrepreneurial instincts.
- To forge collaborative research between academia and industry for seamless transfer of knowledge resulting in sponsored projects and consultancy.
- To inculcate responsibility through knowledge sharing and designing innovative computing solutions that benefits the society-at-large.
- To inculcate environmental sense with research, industry and community to set high standards in academic excellence and in fulfilling societal responsibilities.

PROGRAM EDUCATIONAL OBJECTIVES

The following Program Educational Objectives (PEOs) have been adopted to attain the vision and mission of the Institution and the Department of Computer Science and Engineering.

- **PEO1:** Graduates will build successful career in software related industries or will pursue higher studies in elite institutions in India/ abroad.
- **PEO2:** Graduates will apply the computer engineering principles learnt to provide solutions for the challenging problems in their profession.
- **PEO3:** Graduates will adapt the challenging work environment through life-long learning for the continuous professional development.
- **PEO4:** Graduates will articulate to work in the teams and exhibit high level of professionalism and ethical standards.

LIST POs

PO1	Engineering Knowledge: Apply the knowledge of mathematics science, engineering fundamentals and an engineering specialization to the solution of complex engineering problem
PO2	Problem Analysis: Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural science and

	engineering, sciences.
PO3	Design/Development of Solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety and the cultural, societal, and environmental considerations.
PO4	Conduct investigations of complex problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.
PO5	Modern tool usage: Create, select and apply appropriate techniques, resources and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.
PO6	The engineer and society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice
PO7	Environment and sustainability: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.
PO8	Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice
PO9	Individual and team work: Function effectively as an individual and as a member or leader in diverse teams and in multidisciplinary settings
PO10	Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.
PO11	Project Management and finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.
PO12	Life-long learning: Recognize the need for and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

LIST PSOs

At the end of 4 years course period, Computer Science and Engineering graduates at NGIT will be able to:

PSO1	Shall have the ability to find or create opportunities to design and develop appropriate Computer Science solutions for improving the living standards of the people.
PSO2	Shall have expertise in few of the trending technologies like Python, Machine Learning, Deep Learning, Internet of Things (IOT), Data Science, Full stack development, Social Networks, Cyber Security, Big Data, Mobile Apps, CRM, ERP etc.

GENERAL LABORATORY INSTRUCTIONS

1. Students are advised to come to the laboratory at least 5 minutes before (to starting time), those who come after 5 minutes will not be allowed into the lab.
2. Plan your task properly much before to the commencement, come prepared to the lab with the program / experiment details.
3. Student should enter into the laboratory with:
 - a. Laboratory observation notes with all the details (Problem statement, Aim, Algorithm, Procedure, Program, Expected Output, etc.,) filled in for the lab session.
 - b. Laboratory Record updated up to the last session experiments.
 - c. Formal dress code and Identity card.
4. Sign in the laboratory login register, write the TIME-IN, and occupy the computer system allotted to you by the faculty.
5. Execute your task in the laboratory, and record the results / output in the lab observation note book, and get certified by the concerned faculty.
6. All the students should be polite and cooperative with the laboratory staff, must maintain the discipline and decency in the laboratory.
7. Computer labs are established with sophisticated and high end branded systems, which should be utilized properly.
8. Students / Faculty must keep their mobile phones in SWITCHED OFF mode during the lab sessions. Misuse of the equipment, misbehaviors with the staff and systems etc., will attract severe punishment.
9. Students must take the permission of the faculty in case of any urgency to go out. If anybody found loitering outside the lab / class without permission during working hours will be treated seriously and punished appropriately.
10. Students should SHUT DOWN the computer system before he/she leaves the lab after completing the task (experiment) in all aspects. He/she must ensure the system / seat is kept properly.

Head of the Department

Principal

CODE OF CONDUCT FOR THE LABORATORY

- All students must observe the dress code while in the laboratory
- Footwear is NOT allowed
- Foods, drinks and smoking are NOT allowed
- All bags must be left at the indicated place
- The lab timetable must be strictly followed
- Be PUNCTUAL for your laboratory session
- All programs must be completed within the given time
- Noise must be kept to a minimum
- Workspace must be kept clean and tidy at all time
- All students are liable for any damage to system due to their own negligence
- Students are strictly PROHIBITED from taking out any items from the laboratory
- Report immediately to the lab programmer if any damages to equipment

BEFORE LEAVING LAB:

- Arrange all the equipment and chairs properly.
- Turn off / shut down the systems before leaving.
- Please check the laboratory notice board regularly for updates.

Lab In – charge



NEIL GOGTE INSTITUTE OF TECHNOLOGY

A unit of Keshav Memorial Technical Educational Society (KMTES)
(Approved by AICTE, New Delhi & Affiliated to Osmania University, Hyderabad)
D.No:10TC-111, Kachivanisingaram (V), Uppal, Hyderabad-500088, Telangana.

List of COURSE OUTCOMES

DATA STRUCTURES AND ALGORITHMS LAB (PC 351 CS)

Course Code	Course Outcome	Taxonomy Level
CO1	Interpret the principles of Assembly Language Programming, instruction set in developing microprocessor based applications.	L2
CO2	Develop Applications such as: 8-bit Addition, Multiplication, Division, array operations, swapping, negative and positive numbers.	L3
CO3	Analyse the interfaces like serial ports, digital-to-analog Converters and analog-todigital converters etc.	L4
CO4	Build interfaces of Input-output and other units like stepper motor with 8085. Analyse the function of traffic light controller	L4

MAPPING OF COs WITH POs & PSOs (Curriculum):

Correlation Level: High – 3; Medium – 2; Low – 1

	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12	PSO1	PSO2
CO1	3	3	3	3	3									
CO2	3	3	3	3	3									
CO3	3	3	3	3	3									
CO4	3	3	3	3	3			1	3	3				

CSE: SEMESTER – IV

S. No.	Course Code	Course Title	Scheme of Instruction				Scheme of Examination			Credits
			L	T	P/D	Contact Hrs/Wk	CIE	SEE	Duration in Hrs	
Theory Courses										
1	HS 104 EG	Effective Technical Communication in English	3	-	-	3	30	70	3	3
2	HS 105 CM	Finance and Accounting	3	1	-	4	30	70	3	3
3	BS 207 MT	Mathematics – III (Probability & Statistics)	3	-	-	3	30	70	3	3
4	ES 305 EC	Signals and Systems	3	-	-	3	30	70	3	3
5	PC 401 CS	Operating Systems	3	-	-	3	30	70	3	3
6	PC 402 CS	Computer Organization	3	1	-	4	30	70	3	3
7	PC 403 CS	Database Management Systems	3	1	-	4	30	70	3	3
Practical/ Laboratory Courses										
8	PC 451 CS	Computer Organization Lab	-	-	2	2	25	50	3	1
9	PC 452 CS	Operating Systems Lab	-	-	2	2	25	50	3	1
10	PC 453 CS	Database Management Systems Lab	-	-	2	2	25	50	3	1
Total			21	3	06	30	285	640		24

Practical / Laboratory Courses

Computer Organization Lab

PC 451 CS

Instruction: 2 periods per week

CIE: 25 marks

Credits: 1

Duration of SEE: 3 hours

SEE: 50 marks

Objectives:

The objectives of the course are to impart knowledge of:

1. To become familiar with the architecture and Instruction set of Intel 8085 microprocessor.
2. To provide practical hands on experience with Assembly Language Programming.
3. To familiarize the students with interfacing of various peripheral devices with 8085 microprocessors.

Outcomes:

After the completion of the course, the student will be able to:

1. Interpret the principles of Assembly Language Programming, instruction set in developing microprocessor based applications.
2. Develop Applications such as: 8-bit Addition, Multiplication, Division, array operations, swapping, negative and positive numbers.
3. Analyse the interfaces like serial ports, digital-to-analog Converters and analog-to-digital converters etc.
4. Build interfaces of Input-output and other units like stepper motor with 8085. Analyse the function of traffic light controller.

List of Programs:

1.	Tutorials on 8085 Programming.	
2.	Interfacing and programming of 8255. (E.g. traffic light controller).	
3.	Interfacing and programming of 8254.	
4.	Interfacing and programming of 8279.	
5.	A/D and D/A converter interface.	
6.	Stepper motor interface.	
7.	Display interface	

Note: Adequate number of programs covering all the instructions of 8085 instruction set should be done on the 8085 microprocessor trainer kit.



NEIL GOGTE INSTITUTE OF TECHNOLOGY

A unit of Keshav Memorial Technical Educational Society (KMTES)
(Approved by AICTE, New Delhi & Affiliated to Osmania University, Hyderabad)
D.No:10TC-111, Kachivanisingaram (V), Uppal, Hyderabad-500088, Telangana.

LIST OF EXPERIMENTS

Exp. No.	Name of the Experiment	Date of Conducted	Date of Submission	Faculty Signature
1.				
2				
3				
4				
5.				
6.				
7.				
8.				
9.				
10.				
11.				
12.				
13.				
14.				
15.				
	ADDITIONAL EXPERIMENTS			
1				
2				

Introduction

ARCHITECTURE OF INTEL 8085 MICROPROCESSOR

Features of 8085

- Intel 8085 is an 8-bit, NMOS microprocessor.
- It is a 40 pin I.C. package fabricated on a single LSI chip.
- The Intel 8085 uses a single +5Vd.c. supply for its operation
- Its clock speed is about 3 MHz
- The clock cycle is of 320 ns.
- It has 80 basic instructions and 256 opcodes.

Figure 1.2.1 shows the functional block diagram of Intel 8085. It consists of three main sections:

- [1] Arithmetic and Logic Unit (ALU)
- [2] Timing & Control Unit
- [3] Set of registers.

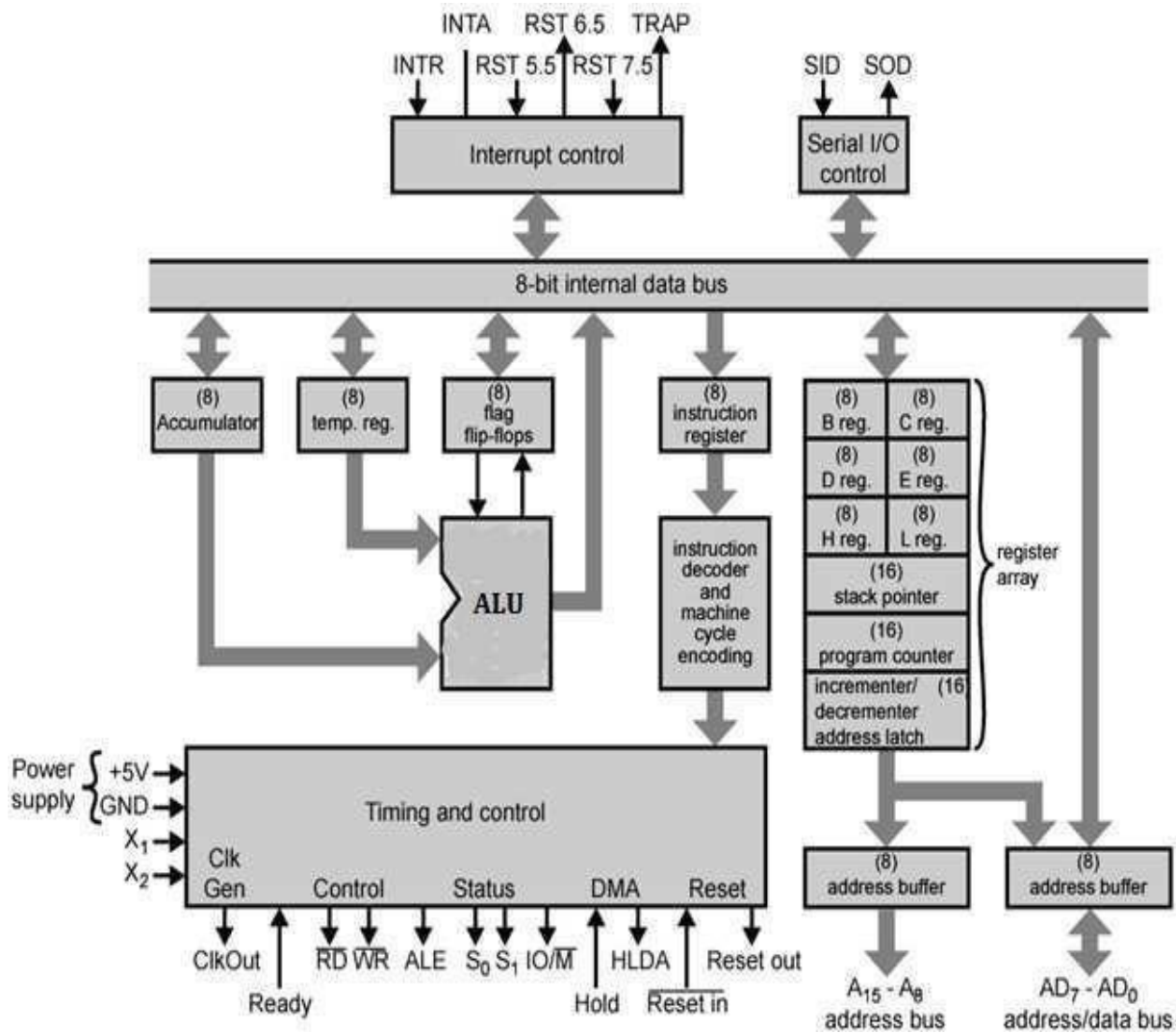


Figure 1.2.1 Architecture 8085 processor

1. ALU (Arithmetic & Logic Unit):

As the name suggest, ALU i.e arithmetic & logic unit performs the following arithmetic & logical opearions:

- i. Addition
- ii. Subtraction
- iii. Logical AND
- iv. Logical OR
- v. Logical EXCLUSIVE OR
- vi. Complement (logical NOT)
- vii. Increment (add 1)
- viii. Decrement (Subtract 1)
- ix. Left shift, Rotate left, Rorate Right
- x. Clear etc.

2. Timing & Control Unit:

- It generates timing & control signals which are necessary for the execution of instructions.
- It controls data flow between CPU and peripherals including memory.
- It provides status, control and timing signals which are required for the operation of memory and I/O devices.
- It controls entire operations of the microprocessor and peripherals connected to it.
- Thus it is seen that the control unit of the CPU acts as the brain of the computer system.

3. Set of Registers:

- Fig 1.2.1 shows the various registers of Intel 8085 which are used by the microprocessor for temporary storage and manipulation of data and intrsuctions.
- Intel 8085 microprocessor has the following registers:
 - i. One 8-bit Accumulator (ACC) i.e register A
 - ii. Six 8-bit general purpose registers. These are B, C, D, E, H & L
 - iii. One 16-bit stack pointer, SP
 - iv. One 16-bit program Counter, PC
 - v. Intruction register

vi. Temporary register

- In addition to the above mentioned register the 8085 microprocessor contains a set of five flipflops which serve as flag (or status flags). A flag is a flipflop which indicates some condition which arises after execution of an arithmetic or logical instruction.

Accumulator:

- The accumulator (register A) is an 8-bit register associated with ALU.
- It is used to hold one of the operands of an arithmetic or logical operation.
- It serves as one input to the ALU. The other operand for an arithmetic or logical operation may be stored either in the memory or in one of the general purpose registers.
- Final result of an arithmetic or logical operation is placed in the accumulator.

General Purpose Registers

- The 8085 microprocessor contains six 8-bit general purpose registers. They are B, C, D, E, H and L.
- To hold 16-bit data a combination of two 8-bit registers can be employed which known as register pairs.
- The valid register pairs in 8085 are B-C, D-E and H-L. the programmer cannot form the register pair by selecting any two registers of his choice.
- The H-L pair is used to act as memory pointer and for this purpose it holds the 16-bit address of a memory location.
- The general purpose registers and the accumulator are accessible to programmer. He can store data in these registers during writing his program.

Program Counter (PC):

- It is a 16-bit special purpose register used to hold the memory address of the next instruction which is to be executed.
- It keeps the track of memory addresses of the instructions in the program while they are being executed.

The microprocessor increments the content of the program counter during the execution of an instruction so that it points to the address of the next instruction in the program at the end of the execution of an instruction.

Stack Pointer (SP):

- It is a 16-bit special purpose register.
- “Stack is a sequence of memory location set aside by the programmer to store/retrieve the content of accumulator, flags, program counter and general purpose registers during the execution of a program.
- Any portion of the memory can be used as stack. Since the stack works on LIFO (last in first out) principle, its operation is faster compared normal store/ retrieve of memory locations.
- “The SP holds the address of the top element of data stored in the stack.
- “The stack is defined and stack pointer is initialized by the programmer at the beginning of a program which needs stack operation. Stack is also used by the microprocessor. For example, it stores the contents of program counter when it jumps to a subroutine using CALL instruction.

Instruction Register:

- The instruction register holds the opcode (operation code or instruction code) of the instruction which is being decoded and executed.

Temporary Register:

- It is an 8-bit register associated with ALU. It holds data during an arithmetic/logical operation.
- It is used by microprocessor.
- It is not accessible to programmer.

Flags (Program Status Word i.e. PSW):

- The Intel 8085 microprocessor contains five flip-flops to serve as status flags.
- The flip-flops are set or reset according to the conditions which arises during an arithmetic or logical operation.
- The five status flags of Intel 8085 are:

Carry flag (CS)

Parity Flag (P)

Auxiliary Carry Flag (AC)

Zero Flag (Z)

Sign Flag (S)

- In fig 1.2.2, five bits indicates the five status flags and three bits are undefined. “The combination of these 8-bits is called Program Status Word (PSW)”. PSW and the accumulator are treated as a 16-bit unit for stack operation.

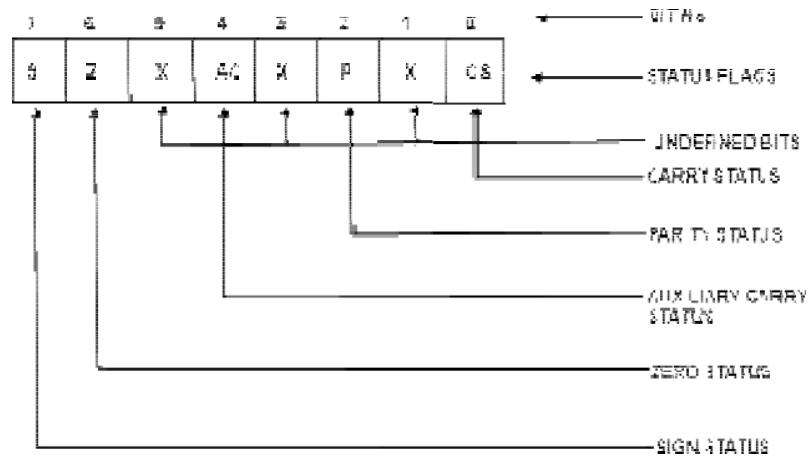


Figure 1.2.2 Flag Register

Carry Flag (CS):

- After execution of an arithmetic instruction if carry is produced, the carry flag CS is set to 1. Otherwise it is 0.
- The carry flag is set or reset in case of addition as well as subtraction.
- After the addition of two 8-bit numbers, if the sum is larger than 8-bits, a carry is produced; and the carry flag is set to 1.
- In case of subtraction, if borrow occurs, the carry flag is set to 1.
- The carry flag holds carry out of the most significant bit resulting from the execution of an arithmetic operation.

Parity Flag (P):

- The parity flag P is set to 1, if the result of an arithmetic or logical operation contains even number of 1s.

It is reset i.e it is 0, if the result contains odd number of 1s

Auxiliary Carry Flag (AC):

- ☐ The auxiliary carry flag AC hold carry out of the bit number 3 to the bit number 4 resulting

from the execution of an arithmetic operation.

- The counting of bits starts from 0, and hence Bit No.3 is actually the fourth bit from the least significant bit.

Zero Flag (Z):

- The zero status flag Z is set to 1, if the result of an arithmetic or logical operation is Zero otherwise it is set to 0.

Sign Flag (S):

- The sign flag S is set to 1, if the result of an arithmetic or logical operation is negative. If the result is positive, the sign flag is set to 0.
- The sign flag has significance only when signed arithmetic operation is performed. To represent a signed number the most significant bit is reserved by the programmer to represent the sign of a number.
- In other words, the MSB is used as a sign bit.
- If the number is negative, the sign bit is 1. For positive, sign bit is 0.
- In case of 8-bit sign operation, the remaining 7 bits are used to represent the magnitude of a number.

Data and Address bus:

- The Intel 8085 is an 8-bit microprocessor. Its data bus is 8-bit wide and hence, 8bit of data can be transmitted in parallel from or to the microprocessor.
- The Intel 8085 requires a 16-bit wide address bus as the memory addresses are of 16 bits.
- The 8 most significant bits of the address are transmitted by the address bus, a-bus (pins A8 to A15).
- The least significant bits of address are transmitted by address/ data bus, AD-bus (pins AD0 to AD7).
- The address/ data bus transmits data and address at different moments. At a particular moment it transmits either data or address. Thus AD-bus operates in time shared mode.

This technique is known as multiplexing. First all 16-bit memory address is transmitted by the microprocessor; the 8 MSBs of the address on A-bus and the 8 LSBs of the address on

AD- bus. Thus effective width of address bus becomes 16 bit wide.

INSTRUCTION SET AND EXECUTION IN 8085

Based on the design of the ALU and decoding unit, the microprocessor manufacturer provides instruction set for every microprocessor. The instruction set consists of both machine code and mnemonics.

An instruction is a binary pattern designed inside a microprocessor to perform a specific function. The entire group of instructions that a microprocessor supports is called instruction set. Microprocessor instructions can be classified based on the parameters such functionality, length and operand addressing.

Classification based on functionality:

- I. **Data transfer operations:** This group of instructions copies data from source to destination. The content of the source is not altered.
- II. **Arithmetic operations:** Instructions of this group perform operations like addition, subtraction, increment & decrement. One of the data used in arithmetic operation is stored in accumulator and the result is also stored in accumulator.
- III. **Logical operations:** Logical operations include AND, OR, EXOR, NOT. The operations like AND, OR and EXOR uses two operands, one is stored in accumulator and other can be any register or memory location. The result is stored in accumulator. NOT operation requires single operand, which is stored in accumulator.
- IV. **Branching operations:** Instructions in this group can be used to transfer program sequence from one memory location to another either conditionally or unconditionally.
- V. **Machine control operations:** Instruction in this group control execution of other instructions and control operations like interrupt, halt etc.

Classification based on length:

- I. **One-byte instructions:** Instruction having one byte in machine code. Examples are depicted in Table 2.
- I. **Two-byte instructions:** Instruction having two byte in machine code. Examples are depicted in Table 3
- II. **Three-byte instructions:** Instruction having three byte in machine code. Examples are depicted in Table 4.

Table 2 Examples of one byte instructions

Opco de	Opera nd	Machine code/Hex code
MOV	A, B	78
ADD	M	86

Table 3 Examples of two byte instructions

Opcode	Operand	Machine code/Hex code	Byte description
MVI	A, 7FH	3E	First byte
		7F	Second byte
ADI	0FH	C6	First byte
		0F	Second byte

Table 4 Examples of three byte instructions

Opcode	Operand	Machine code/Hex code	Byte description
JMP	9050H	C3	First byte
		50	Second byte
		90	Third byte
LDA	8850H	3A	First byte
		50	Second byte
		88	Third byte

Addressing Modes in Instructions:

The process of specifying the data to be operated on by the instruction is called addressing. The various formats for specifying operands are called addressing modes. The 8085 has the following five types of addressing:

- I. Immediate addressing
- II. Memory direct addressing
- III. Register direct addressing
- IV. Indirect addressing
- V. Implicit addressing

Immediate Addressing:

In this mode, the operand given in the instruction - a byte or word – transfers to the destination register or memory location.

Ex: MVI A, 9AH

- The operand is a part of the instruction.
- The operand is stored in the register mentioned in the instruction.

Memory Direct Addressing:

Memory direct addressing moves a byte or word between a memory location and register. The memory location address is given in the instruction.

Ex: LDA 850FH

This instruction is used to load the content of memory address 850FH in the accumulator.

Register Direct Addressing:

Register direct addressing transfer a copy of a byte or word from source register to destination register.

Ex: MOV B, C

It copies the content of register C to register B.

Indirect Addressing:

Indirect addressing transfers a byte or word between a register and a memory location.

Ex: MOV A, M

Here the data is in the memory location pointed to by the contents of HL pair. The data is moved to the accumulator.

Implicit Addressing

In this addressing mode the data itself specifies the data to be operated upon.

Ex: CMA

The instruction complements the content of the accumulator. No specific data or operand is mentioned in the instruction

INSTRUCTION SET OF 8085

Data Transfer Instructions:

Opcode	Operand	Description
Copy from source to destination		
MOV	Rd, Rs M, Rs Rd, M	This instruction copies the contents of the source register into the destination register; the contents of the source register are not altered. If one of the operands is a memory location, its location is specified by the contents of the HL registers. Example: MOV B, C or MOV B, M
Move immediate 8-bit		
MVI	Rd, data M, data	The 8-bit data is stored in the destination register or memory. If the operand is a memory location, its location is specified by the contents of the HL registers. Example: MVI B, 57 or MVI M, 57
Load accumulator		
LDA	16-bit address	The contents of a memory location, specified by a 16-bit address in the operand, are copied to the accumulator. The contents of the source are not altered. Example: LDA 2034 or LDA XYZ
Load accumulator indirect		
LDAX	B/D Reg. pair	The contents of the designated register pair point to a memory location. This instruction copies the contents of that memory location into the accumulator. The contents of either the register pair or the memory location are not altered. Example: LDAX B
Load register pair immediate		
LXI	Reg. pair, 16-bit data	The instruction loads 16-bit data in the register pair designated in the operand. Example: LXI H, 2034
Load H and L registers direct		
LHLD	16-bit address	The instruction copies the contents of the memory location pointed out by the 16-bit address into register L and copies the contents of the next memory location into register H. The contents of source memory locations are not altered. Example: LHLD 2040

Store accumulator direct

STA 16-bit address

The contents of the accumulator are copied into the memory location specified by the operand. This is a 3-byte instruction, the second byte specifies the low-order address and the third byte specifies the high-order address.

Example: STA 4350 or STA XYZ

Store accumulator indirect

STAX Reg. pair

The contents of the accumulator are copied into the memory location specified by the contents of the operand (register pair). The contents of the accumulator are not altered.

Example: STAX B

Store H and L registers direct

SHLD 16-bit address

The contents of register L are stored into the memory location specified by the 16-bit address in the operand and the contents of H register are stored into the next memory location by incrementing the operand. The contents of registers HL are not altered. This is a 3-byte instruction, the second byte specifies the low-order address and the third byte specifies the high-order address.

Example: SHLD 2470

Exchange H and L with D and E

XCHG none

The contents of register H are exchanged with the contents of register D, and the contents of register L are exchanged with the contents of register E.

Example: XCHG

Copy H and L registers to the stack pointer

SPHL none

The instruction loads the contents of the H and L registers into the stack pointer register, the contents of the H register provide the high-order address and the contents of the L register provide the low-order address. The contents of the H and L registers are not altered.

Example: SPHL

Exchange H and L with top of stack

XTHL none

The contents of the L register are exchanged with the stack location pointed out by the contents of the stack pointer register. The contents of the H register are exchanged with the next stack location (SP+1); however, the contents of the stack pointer register are not altered.

Example: XTHL

Push register pair onto stack

PUSH Reg. pair

The contents of the register pair designated in the operand are copied onto the stack in the following sequence. The stack pointer register is decremented and the contents of the high-order register (B, D, H, A) are copied into that location. The stack pointer register is decremented again and the contents of the low-order register (C, E, L, flags) are copied to that location.

Example: PUSH B or PUSH A

Pop off stack to register pair

POP Reg. pair

The contents of the memory location pointed out by the stack pointer register are copied to the low-order register (C, E, L, status flags) of the operand. The stack pointer is incremented by 1 and the contents of that memory location are copied to the high-order register (B, D, H, A) of the operand. The stack pointer register is again incremented by 1.

Example: POP H or POP A

Output data from accumulator to a port with 8-bit address

OUT 8-bit port address

The contents of the accumulator are copied into the I/O port specified by the operand.

Example: OUT 87

Input data to accumulator from a port with 8-bit address

IN 8-bit port address

The contents of the input port designated in the operand are read and loaded into the accumulator.

Example: IN 82



NEIL GOGTE INSTITUTE OF TECHNOLOGY

A unit of Keshav Memorial Technical Educational Society (KMTES)
(Approved by AICTE, New Delhi & Affiliated to Osmania University, Hyderabad)
D.No:10TC-111, Kachivanisingaram (V), Uppal, Hyderabad-500088, Telangana.

Arithmetic Instructions:

Opcode	Operand	Description
Add register or memory to accumulator		
ADD	R M	The contents of the operand (register or memory) are added to the contents of the accumulator and the result is stored in the accumulator. If the operand is a memory location, its location is specified by the contents of the HL registers. All flags are modified to reflect the result of the addition. Example: ADD B or ADD M
Add register to accumulator with carry		
ADC	R M	The contents of the operand (register or memory) and the Carry flag are added to the contents of the accumulator and the result is stored in the accumulator. If the operand is a memory location, its location is specified by the contents of the HL registers. All flags are modified to reflect the result of the addition. Example: ADC B or ADC M
Add immediate to accumulator		
ADI	8-bit data	The 8-bit data (operand) is added to the contents of the accumulator and the result is stored in the accumulator. All flags are modified to reflect the result of the addition. Example: ADI 45
Add immediate to accumulator with carry		
ACI	8-bit data	The 8-bit data (operand) and the Carry flag are added to the contents of the accumulator and the result is stored in the accumulator. All flags are modified to reflect the result of the addition. Example: ACI 45
Add register pair to H and L registers		
DAD	Reg. pair	The 16-bit contents of the specified register pair are added to the contents of the HL register and the sum is stored in the HL register. The contents of the source register pair are not altered. If the result is larger than 16 bits, the CY flag is set. No other flags are affected. Example: DAD H

Subtract register or memory from accumulator

SUB	R	The contents of the operand (register or memory) are subtracted from the contents of the accumulator, and the result is stored in the accumulator. If the operand is a memory location, its location is specified by the contents of the HL registers. All flags are modified to reflect the result of the subtraction. Example: SUB B or SUB M
	M	

Subtract source and borrow from accumulator

SBB	R	The contents of the operand (register or memory) and the Borrow flag are subtracted from the contents of the accumulator and the result is placed in the accumulator. If the operand is a memory location, its location is specified by the contents of the HL registers. All flags are modified to reflect the result of the subtraction. Example: SBB B or SBB M
	M	

Subtract immediate from accumulator

SUI	8-bit data	The 8-bit data (operand) is subtracted from the contents of the accumulator and the result is stored in the accumulator. All flags are modified to reflect the result of the subtraction. Example: SUI 45
-----	------------	--

Subtract immediate from accumulator with borrow

SBI	8-bit data	The 8-bit data (operand) and the Borrow flag are subtracted from the contents of the accumulator and the result is stored in the accumulator. All flags are modified to reflect the result of the subtraction. Example: SBI 45
-----	------------	---

Increment register or memory by 1

INR	R	The contents of the designated register or memory) are incremented by 1 and the result is stored in the same place. If the operand is a memory location, its location is specified by the contents of the HL registers. Example: INR B or INR M
	M	

Increment register pair by 1

INX	R	The contents of the designated register pair are incremented by 1 and the result is stored in the same place. Example: INX H
-----	---	---

Decrement register or memory by 1

DCR R The contents of the designated register or memory are
 M decremented by 1 and the result is stored in the same place. If
 the operand is a memory location, its location is specified by
 the contents of the HL registers.
Example: DCR B or DCR M

Decrement register pair by 1

DCX R The contents of the designated register pair are decremented
 by 1 and the result is stored in the same place.
Example: DCX H

Decimal adjust accumulator

DAA none The contents of the accumulator are changed from a binary
 value to two 4-bit binary coded decimal (BCD) digits. This is
 the only instruction that uses the auxiliary flag to perform the
 binary to BCD conversion, and the conversion procedure is
 described below. S, Z, AC, P, CY flags are altered to reflect
 the results of the operation.

If the value of the low-order 4-bits in the accumulator is greater than 9 or if AC flag is set, the instruction adds 6 to the low-order four bits.

If the value of the high-order 4-bits in the accumulator is greater than 9 or if the Carry flag is set, the instruction adds 6 to the high-order four bits.

Example: DAA

BRANCHING INSTRUCTIONS

Opcode	Operand	Description
Jump unconditionally		
JMP	16-bit address	The program sequence is transferred to the memory location specified by the 16-bit address given in the operand. Example: JMP 2034 or JMP XYZ

Jump conditionally

Operand: 16-bit address

The program sequence is transferred to the memory location specified by the 16-bit address given in the operand based on the specified flag of the PSW as described below.
Example: JZ 2034 or JZ XYZ

Opcode	Description	Flag Status
JC	Jump on Carry	CY = 1
JNC	Jump on no Carry	CY = 0
JP	Jump on positive	S = 0
JM	Jump on minus	S = 1
JZ	Jump on zero	Z = 1
JNZ	Jump on no zero	Z = 0
JPE	Jump on parity even	P = 1
JPO	Jump on parity odd	P = 0

Unconditional subroutine call
CALL 16-bit address

The program sequence is transferred to the memory location specified by the 16-bit address given in the operand. Before the transfer, the address of the next instruction after CALL (the contents of the program counter) is pushed onto the stack.
Example: CALL 2034 or CALL XYZ

Call conditionally

Operand: 16-bit address

The program sequence is transferred to the memory location specified by the 16-bit address given in the operand based on the specified flag of the PSW as described below. Before the transfer, the address of the next instruction after the call (the contents of the program counter) is pushed onto the stack.
Example: CZ 2034 or CZ XYZ

Opcode	Description	Flag Status
CC	Call on Carry	CY = 1
CNC	Call on no Carry	CY = 0
CP	Call on positive	S = 0
CM	Call on minus	S = 1
CZ	Call on zero	Z = 1
CNZ	Call on no zero	Z = 0
CPE	Call on parity even	P = 1
CPO	Call on parity odd	P = 0

4.

Return from subroutine unconditionally

RET none

The program sequence is transferred from the subroutine to the calling program. The two bytes from the top of the stack are copied into the program counter, and program execution begins at the new address.
Example: RET

Return from subroutine conditionally

Operand: none

The program sequence is transferred from the subroutine to the calling program based on the specified flag of the PSW as described below. The two bytes from the top of the stack are copied into the program counter, and program execution begins at the new address.
Example: RZ

Opcode	Description	Flag Status
RC	Return on Carry	CY = 1
RNC	Return on no Carry	CY = 0
RP	Return on positive	S = 0
RM	Return on minus	S = 1
RZ	Return on zero	Z = 1
RNZ	Return on no zero	Z = 0
RPE	Return on parity even	P = 1
RPO	Return on parity odd	P = 0

Load program counter with HL contents

PCHL none The contents of registers H and L are copied into the program counter. The contents of H are placed as the high-order byte and the contents of L as the low-order byte.
Example: PCHL

Restart

RST 0-7 The RST instruction is equivalent to a 1-byte call instruction to one of eight memory locations depending upon the number. The instructions are generally used in conjunction with interrupts and inserted using external hardware. However these can be used as software instructions in a program to transfer program execution to one of the eight locations. The addresses are:

Instruction	Restart Address
RST 0	0000H
RST 1	0008H
RST 2	0010H
RST 3	0018H
RST 4	0020H
RST 5	0028H
RST 6	0030H
RST 7	0038H

The 8085 has four additional interrupts and these interrupts generate RST instructions internally and thus do not require any external hardware. These instructions and their Restart addresses are:

Interrupt	Restart Address
TRAP	0024H
RST 5.5	002CH
RST 6.5	0034H
RST 7.5	003CH

LOGICAL INSTRUCTIONS

Opcode	Operand	Description
--------	---------	-------------

Compare register or memory with accumulator

CMP	R	The contents of the operand (register or memory) are compared with the contents of the accumulator. Both contents are preserved. The result of the comparison is shown by setting the flags of the PSW as follows: if (A) < (reg/mem): carry flag is set, s=1 if (A) = (reg/mem): zero flag is set, s=0 if (A) > (reg/mem): carry and zero flags are reset, s=0 Example: CMP B or CMP M
	M	

Compare immediate with accumulator

CPI	8-bit data	The second byte (8-bit data) is compared with the contents of the accumulator. The values being compared remain unchanged. The result of the comparison is shown by setting the flags of the PSW as follows: if (A) < data: carry flag is set, s=1 if (A) = data: zero flag is set, s=0 if (A) > data: carry and zero flags are reset, s=0 Example: CPI 89
-----	------------	--

Logical AND register or memory with accumulator

ANA	R	The contents of the accumulator are logically ANDed with the contents of the operand (register or memory), and the result is placed in the accumulator. If the operand is a memory location, its address is specified by the contents of HL registers. S, Z, P are modified to reflect the result of the operation. CY is reset. AC is set. Example: ANA B or ANA M
	M	

Logical AND immediate with accumulator

ANI	8-bit data	The contents of the accumulator are logically ANDed with the 8-bit data (operand) and the result is placed in the accumulator. S, Z, P are modified to reflect the result of the operation. CY is reset. AC is set. Example: ANI 86
-----	------------	--



NEIL GOGTE INSTITUTE OF TECHNOLOGY

A unit of Keshav Memorial Technical Educational Society (KMTES)
(Approved by AICTE, New Delhi & Affiliated to Osmania University, Hyderabad)
D.No:10TC-111, Kachivanisingaram (V), Uppal, Hyderabad-500088, Telangana.

Exclusive OR register or memory with accumulator

XRA R
 M

The contents of the accumulator are Exclusive ORed with the contents of the operand (register or memory), and the result is placed in the accumulator. If the operand is a memory location, its address is specified by the contents of HL registers. S, Z, P are modified to reflect the result of the operation. CY and AC are reset.
Example: XRA B or XRA M

Exclusive OR immediate with accumulator

XRI 8-bit data

The contents of the accumulator are Exclusive ORed with the 8-bit data (operand) and the result is placed in the accumulator. S, Z, P are modified to reflect the result of the operation. CY and AC are reset.
Example: XRI 86

Logical OR register or memory with accumulator

ORA R
 M

The contents of the accumulator are logically ORed with the contents of the operand (register or memory), and the result is placed in the accumulator. If the operand is a memory location, its address is specified by the contents of HL registers. S, Z, P are modified to reflect the result of the operation. CY and AC are reset.
Example: ORA B or ORA M

Logical OR immediate with accumulator

ORI 8-bit data

The contents of the accumulator are logically ORed with the 8-bit data (operand) and the result is placed in the accumulator. S, Z, P are modified to reflect the result of the operation. CY and AC are reset.
Example: ORI 86

Rotate accumulator left

RLC none

Each binary bit of the accumulator is rotated left by one position. Bit D7 is placed in the position of D0 as well as in the Carry flag. CY is modified according to bit D7. S, Z, P, AC are not affected.
Example: RLC

Rotate accumulator right

RRC none

Each binary bit of the accumulator is rotated right by one position. Bit D0 is placed in the position of D7 as well as in the Carry flag. CY is modified according to bit D0. S, Z, P, AC are not affected.
Example: RRC



NEIL GOGTE INSTITUTE OF TECHNOLOGY

A unit of Keshav Memorial Technical Educational Society (KMES)
(Approved by AICTE, New Delhi & Affiliated to Osmania University, Hyderabad)
D.No:10TC-111, Kachivanisingaram (V), Uppal, Hyderabad-500088, Telangana.

Rotate accumulator left through carry

RAL none

Each binary bit of the accumulator is rotated left by one position through the Carry flag. Bit D7 is placed in the Carry flag, and the Carry flag is placed in the least significant position D0. CY is modified according to bit D7. S, Z, P, AC are not affected.

Example: RAL

Rotate accumulator right through carry

RAR none

Each binary bit of the accumulator is rotated right by one position through the Carry flag. Bit D0 is placed in the Carry flag, and the Carry flag is placed in the most significant position D7. CY is modified according to bit D0. S, Z, P, AC are not affected.

Example: RAR

Complement accumulator

CMA none

The contents of the accumulator are complemented. No flags are affected.

Example: CMA

Complement carry

CMC none

The Carry flag is complemented. No other flags are affected.

Example: CMC

Set Carry

STC none

The Carry flag is set to 1. No other flags are affected.

Example: STC

CONTROL INSTRUCTIONS

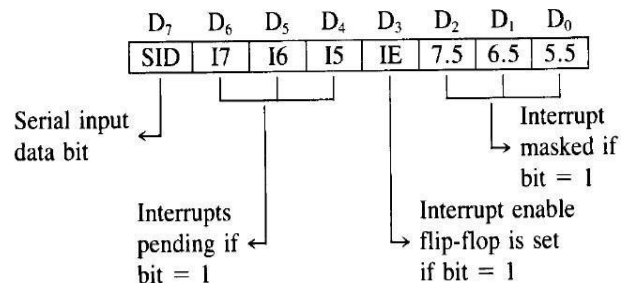
Opcode	Operand	Description
No operation NOP	none	No operation is performed. The instruction is fetched and decoded. However no operation is executed. Example: NOP
Halt and enter wait state HLT	none	The CPU finishes executing the current instruction and halts any further execution. An interrupt or reset is necessary to exit from the halt state. Example: HLT
Disable interrupts DI	none	The interrupt enable flip-flop is reset and all the interrupts except the TRAP are disabled. No flags are affected. Example: DI
Enable interrupts EI	none	The interrupt enable flip-flop is set and all interrupts are enabled. No flags are affected. After a system reset or the acknowledgement of an interrupt, the interrupt enable flip-flop is reset, thus disabling the interrupts. This instruction is necessary to reenables the interrupts (except TRAP). Example: EI

Read interrupt mask

RIM none

This is a multipurpose instruction used to read the status of interrupts 7.5, 6.5, 5.5 and read serial data input bit. The instruction loads eight bits in the accumulator with the following interpretations.

Example: RIM

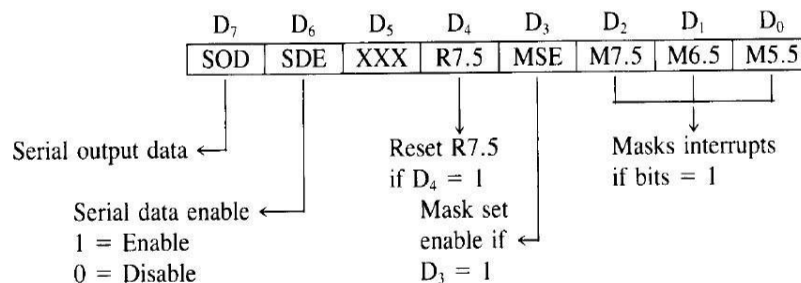


Set interrupt mask

SIM none

This is a multipurpose instruction and used to implement the 8085 interrupts 7.5, 6.5, 5.5, and serial data output. The instruction interprets the accumulator contents as follows.

Example: SIM



- ☐ SOD—Serial Output Data: Bit D7 of the accumulator is latched into the SOD output line and made available to a serial peripheral if bit D6 = 1.
- ☐ SDE—Serial Data Enable: If this bit = 1, it enables the serial output. To implement serial output, this bit needs to be enabled.
- ☐ XXX—Don't Care
- ☐ R7.5—Reset RST 7.5: If this bit = 1, RST 7.5 flip-flop is reset. This is an additional control to reset RST 7.5.
- ☐ MSE—Mask Set Enable: If this bit is high, it enables the functions of bits D2, D1, D0. This is a master control over all the interrupt masking bits. If this bit is low, bits D2, D1, and D0 do not have any effect on the masks.
- ☐ M7.5—D2 = 0, RST 7.5 is enabled.
= 1, RST 7.5 is masked or disabled.
- ☐ M6.5—D1 = 0, RST 6.5 is enabled.
= 1, RST 6.5 is masked or disabled.
- ☐ M5.5—D0 = 0, RST 5.5 is enabled.
= 1, RST 5.5 is masked or disabled.

INTEL 8085 Instructions (Alphabetical order)

Hex	Mnemonic	Hex	Mnemonic	Hex	Mnemonic	Hex	Mnemonic
CE	ACI 8-Bit	3F	CMC	2B	DCX H	01	LXI B, 16-Bit
8F	ADC A	BF	CMP A	3B	DCX SP	11	LXI D, 16-Bit
88	ADC B	B8	CMP B	F3	DI	21	LXI H, 16-Bit
89	ADC C	B9	CMP C	FB	EI	31	LXI SP, 16-Bit
8A	ADC D	BA	CMP D	76	HLT	7F	MOV A,A
8B	ADC E	BB	CMP E	DB	IN 8-Bit	78	MOV A,B
8C	ADC H	BC	CMP H	3C	INR A	79	MOV A,C
8D	ADC L	BD	CMP L	04	INR B	7A	MOV A,D
8E	ADC M	BE	CMP M	0C	INR C	7B	MOV A,E
87	ADD A	D4	CNC 16-Bit	14	INR D	7C	MOV A,H
80	ADD B	C4	CNZ 16-Bit	1C	INR E	7D	MOV A,L
81	ADD C	F4	CP 16-Bit	24	INR H	7E	MOV A,M
82	ADD D	EC	CPE 16-Bit	2C	INR L	47	MOV B,A
83	ADD E	FE	CPI 8-Bit	34	INR M	40	MOV B,B
84	ADD H	E4	CPO 16-Bit	03	INX D	41	MOV B,C
85	ADD L	CC	CZ 16-Bit	13	INX D	42	MOV B,D
86	ADD M	27	DAA	23	INX H	43	MOV B,E
C6	ADI 8-Bit	09	DAD B	33	INX SP	44	MOV B,H
A7	ANA A	19	DAD D	DA	JC 16-Bit	45	MOV B,L
A0	ANA B	29	DAD H	FA	JM 16-Bit	46	MOV B,M
A1	ANA C	39	DAD SP	C3	JMP 16-Bit	4F	MOV C,A
A2	ANA D	3D	DCR A	D2	JNC 16-Bit	48	MOV C,B
A3	ANA E	05	DCR B	C2	JNZ 16-Bit	49	MOV C,C
A4	ANA H	0D	DCR C	F2	JP 16-Bit	4A	MOV C,D
A5	ANA L	15	DCR D	EA	JPE 16-Bit	4B	MOV C,E
A6	ANA M	1D	DCR E	E2	JPO 16-Bit	4C	MOV C,H
E6	ANI 8-Bit	25	DCR H	CA	JZ 16-Bit	4D	MOC C,L
CD	CALL 16-Bit	2D	DCR L	3A	LDA 16-Bit	4E	MOV C,M
DC	CC 16-Bit	35	DCR M	0A	LDAX B	57	MOV D,A
FC	CM 16-Bit	0B	DCX B	1A	LDAX D	50	MOV D,B
2F	CMA	1B	DCX D	2A	LHLD 16-Bit	51	MOV D,C

Hex	Mnemonic	Hex	Mnemonic	Hex	Mnemonic	Hex	Mnemonic
52	MOV D,D	71	MOV M,C	E5	PUSH H	9E	SBB M
53	MOV D,E	72	MOV M,D	F5	PUSH PSW	DE	SBI 8-Bit
54	MOV D,H	73	MOV M,E	17	RAL	22	SHLD 16-Bit
55	MOV D,L	74	MOV M,H	1F	RAR	30	SIM
56	MOV D,M	75	MOV M,L	D8	RC	F9	SPHL
5F	MOV E,A	3E	MVI A, 8-Bit	C9	RET	32	STA 16-Bit
58	MOV E,B	06	MVI B, 8-Bit	20	RIM	02	STAX B
59	MOV E,C	0E	MVI C, 8-Bit	07	RLC	12	STAX D
5A	MOV E,D	16	MVI D, 8-Bit	F8	RM	37	STC
5B	MOV E,E	1E	MVI E, 8-Bit	D0	RNC	97	SUB A
5C	MOV E,H	26	MCI M, 8-Bit	C0	RNZ	90	SUB B
5D	MOV E,L	2E	MVI L, 8-Bit	F0	RP	91	SUB C
5E	MOV E,M	36	MVI M, 8-Bit	E8	RPE	92	SUB D
57	MOV H,A	00	NOP	E0	RPO	93	SUB E
60	MOV H,B	B7	ORA A	0F	RRC	94	SUB H
61	MOV H,C	B0	ORA B	C7	RST 0	95	SUB L
62	MOV H,D	B1	ORA C	CF	RST 1	96	SUB M
63	MOV H,E	B2	ORA D	D7	RST 2	D6	SUI 8-Bit
64	MOV H,H	B3	ORA E	DF	RST 3	EB	XCHG
65	MOV H,L	B4	ORA H	E7	RST 4	AF	XRA A
66	MOV H,M	B5	ORA L	EF	RST 5	A8	XRA B
6F	MOV L,A	B6	ORA M	F7	RST 6	A9	XRA C
68	MOV L,B	F6	ORI 8-Bit	FF	RST 7	AA	XRA D
69	MOV L,C	D3	OUT 8-Bit	C8	RZ	AB	XRA E
6A	MOV L,D	E9	PCHL	9F	SBB A	AC	XRA H
6B	MOV L,E	C1	POP B	98	SBB B	AD	XRA L
6C	MOV L,H	D1	POP D	99	SBB C	AE	XRA M
6D	MOV L,L	E1	POP H	9A	SBB D	EE	XRI 8-Bit
6E	MOV L,M	F1	POP PSW	9B	SBB E	E3	XTHL
77	MOV M,A	C5	PUSH B	9C	SBB H		
70	MOV M,B	D5	PUSH D	9D	SBB L		

NOTE: 1-Byte Instructions: Operand R,M or implicit
2-Byte Instructions: Operand 8-Bit
3-Byte Instructions: Operand 16-Bit

Appendix H

Quick Steps to learn operation with kit

Step I

Procedure to operate kit using assembly language program.

Do connections as per fig. 1

Switch ON power supply (i.e. SMPS)

ANSHUMAN will be display on display board

↓

Press ENTER key

↓

Go to command mode

COMMAND =

↓

Press E

↓

EXPAND =

↓

Press A

↓

2 PASS ASSEMBLER?

↓

Press ENTER key 3 times

↓

File EXIST?

↓

Press 'N' (No)

↓

ARE YOU SURE?

↓

Press ENTER KEY

↓

C =

↓

Press 'W'

↓

0001	ORG	6000H
0002	MVI <i>0C</i>	A, 22H
0003	MVI <i>0C</i>	B, 33H
0004	ADD	B
0005	RST	1

Program to perform
addition of two nos.
using 8085μprocessor

↓

Press Ctrl key + C

↓

C =

↓

Press 'A' Key

↓

Will show message " Save code at DSTN ? Y/N

↓

Press 'Y' (Yes)

↓

C =

↓

Press 'ESC' two times

↓

COMMAND = (Go to command mode)

Execution of the program

↓

Press 'G'

↓

Go to?

↓

Press ENTER key 2 times

↓

BURST ADDR



GIVE 6000H



Press ENTER Key



COMMAND = (Go to command mode)

Checking contents of registers



Press 'S'



SUBSTUT ?



Press ENTER key



MEMORY



Press 'any key' other than 'Enter key' 2 times



IO REGISTER



Press ENTER key



NAME



A = 55 (Result i.e. $22 + 33 = 55$)



(on pressing Enter subsequently all registers contents will display)

Procedure to operate kit using disassembler

Press E

↓
EXPAND =
↓
Press U
↓
Press ENTER key 3 times
↓
STRT
↓
Give 6000
↓
Press ENTER
↓
END
↓
Give 6009
↓
Press ENTER
↓
DSTN
↓
Give 6000
↓
Press ENTER
↓
PRINTER?
↓
Press "CTR+C step by step



NEIL GOGTE INSTITUTE OF TECHNOLOGY

*A unit of Keshav Memorial Technical Educational Society (KMTES)
(Approved by AICTE, New Delhi & Affiliated to Osmania University, Hyderabad)
D.No:10TC-111, Kachivanisingaram (V), Uppal, Hyderabad-500088, Telangana.*

COMPUTER ORGANIZATION LAB

List of Programs : 8085 Programs

1. 8 bit data Addition
2. 8 bit data Subtraction
3. 8 bit data Multiplication
4. 8 bit data Division
5. Largest Element in an array
6. Smallest Element in an array
7. Sorting in Ascending Order
8. Sorting in Descending Order
9. Code Conversions - Decimal to Hexadecimal
10. Code Conversions - Hexadecimal to Decimal
11. BCD Addition
12. BCD Subtraction
13. Introduction to 8255 PPI
14. Interfacing of Stepper Motor (clockwise & anti clockwise)
15. Interfacing of DAC
16. Interfacing of Traffic light controller
17. Interfacing of 8279
18. Display interface



NEIL GOGTE INSTITUTE OF TECHNOLOGY

*A unit of Keshav Memorial Technical Educational Society (KMTES)
(Approved by AICTE, New Delhi & Affiliated to Osmania University, Hyderabad)
D.No:10TC-111, Kachivanisingaram (V), Uppal, Hyderabad-500088, Telangana.*

Exp 1 8 BIT DATA ADDITION

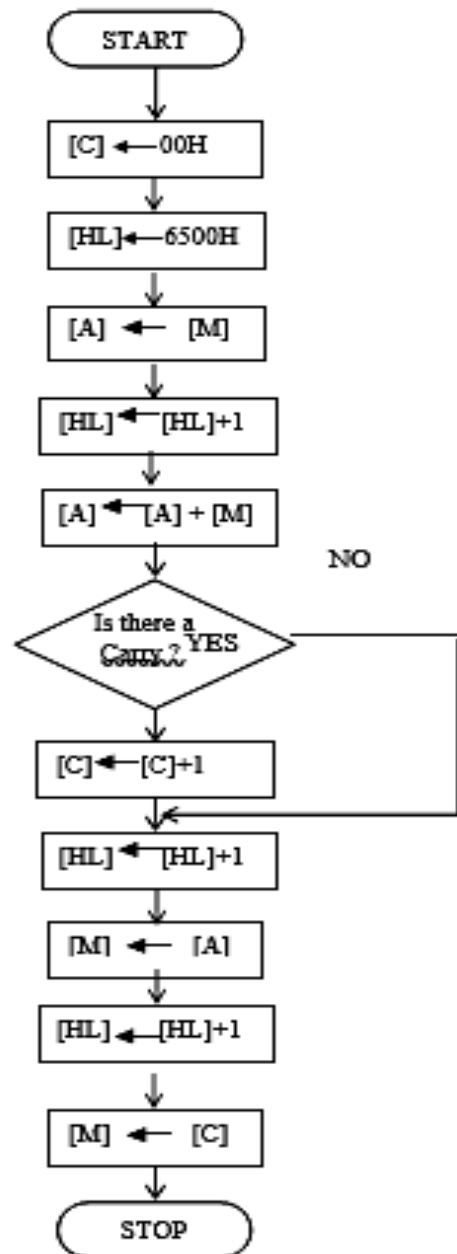
AIM:

To add two 8 bit numbers stored at consecutive memory locations.

ALGORITHM:

1. Initialize memory pointer to data location.
2. Get the first number from memory in accumulator.
3. Get the second number and add it to the accumulator.
4. Store the answer at another memory location.

FLOW CHART:-





NEIL GOGTE INSTITUTE OF TECHNOLOGY

A unit of Keshav Memorial Technical Educational Society (KMTES)
(Approved by AICTE, New Delhi & Affiliated to Osmania University, Hyderabad)
D.No:10TC-111, Kachivanisingaram (V), Uppal, Hyderabad-500088, Telangana.

PROGRAM:

ADDRESS	OPCODE	LABEL	MNEMONICS	OPERAND	COMMENT
6100		START	MVI	C, 00	Clear C reg.
6101					
6102			LXI	H, 6500	Initialize HL reg. to
6103					6500
6104					
6105			MOV	A, M	Transfer first data to
					accumulator
6106			INX	H	Increment HL reg. to
					point next memory
					Location.
6107			ADD	M	Add first number to
					acc. Content.
6108			JNC	L1	Jump to location if
6109					result does not yield
610A					carry.
610B			INR	C	Increment C reg.
610C		L1	INX	H	Increment HL reg. to
					point next memory
					Location.
610D			MOV	M, A	Transfer the result from
					acc. to memory.
610E			INX	H	Increment HL reg. to
					point next memory
					Location.
610F			MOV	M, C	Move carry to memory
6110			HLT		Stop the program



NEIL GOGTE INSTITUTE OF TECHNOLOGY

A unit of Keshav Memorial Technical Educational Society (KMTES)
(Approved by AICTE, New Delhi & Affiliated to Osmania University, Hyderabad)
D.No:10TC-111, Kachivanisingaram (V), Uppal, Hyderabad-500088, Telangana.

OBSERVATION:

INPUT		OUTPUT	
6500		6502	
6501		6503	

RESULT:

Thus the 8 bit numbers stored at 6500 & 6501 are added and the result stored at 6502 and 6503.

Exp 2: 8 BIT DATA SUBTRACTION

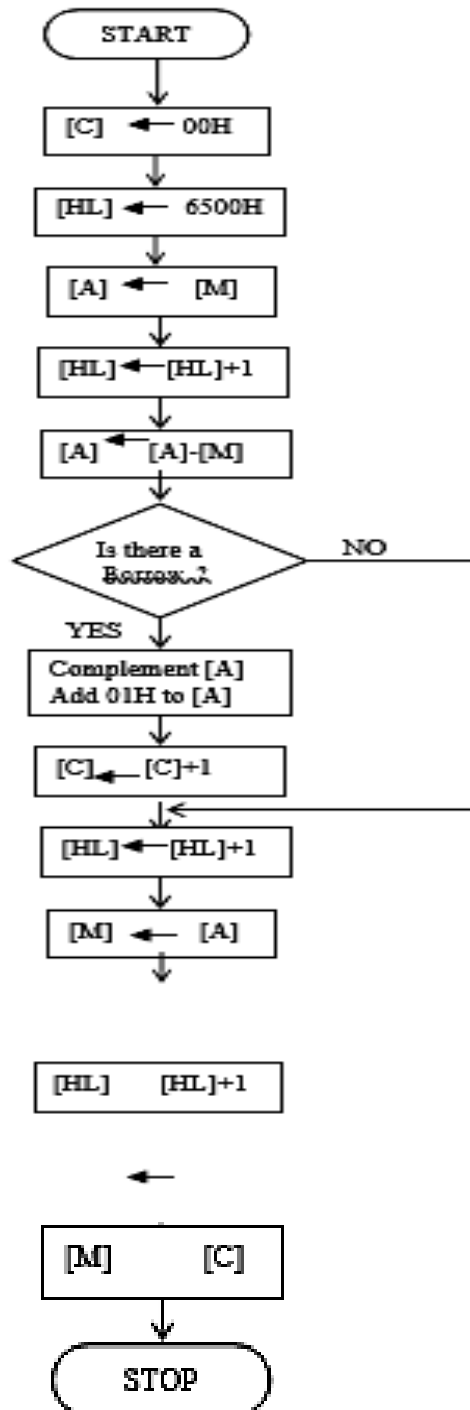
AIM:

To Subtract two 8 bit numbers stored at consecutive memory locations.

ALGORITHM:

1. Initialize memory pointer to data location.
2. Get the first number from memory in accumulator.
3. Get the second number and subtract from the accumulator.
4. If the result yields a borrow, the content of the acc. is complemented and 01H is added to it (2's complement). A register is cleared and the content of that reg. is incremented in case there is a borrow. If there is no borrow the content of the acc. is directly taken as the result.
5. Store the answer at next memory location.

FLOW CHART:





NEIL GOGTE INSTITUTE OF TECHNOLOGY

A unit of Keshav Memorial Technical Educational Society (KMTES)
(Approved by AICTE, New Delhi & Affiliated to Osmania University, Hyderabad)
D.No:10TC-111, Kachivanisingaram (V), Uppal, Hyderabad-500088, Telangana.

PROGRAM:

ADDRESS	OPCODE	LABEL	MNEMONICS	OPERAND	COMMENT
6100		START	MVI	C, 00	Clear C reg.
6101					
6102			LXI	H, 6500	Initialize HL reg. to 6500
6103					
6104					
6105			MOV	A, M	Transfer first data to accumulator
6106			INX	H	Increment HL reg. to point next mem. Location.
6107			SUB	M	Subtract first number from acc. Content.
6108			JNC	L1	Jump to location if result does not yield borrow.
6109					
610A					
610B			INR	C	Increment C reg.
610C			CMA		Complement the Acc. content
610D			ADI	01H	Add 01H to content of acc.
610E					
610F		L1	INX	H	Increment HL reg. to point next mem. Location.
6110			MOV	M, A	Transfer the result from acc. to memory.
6111			INX	H	Increment HL reg. to point next mem. Location.
6112			MOV	M, C	Move carry to mem.
6113			HLT		Stop the program



NEIL GOGTE INSTITUTE OF TECHNOLOGY

A unit of Keshav Memorial Technical Educational Society (KMTES)
(Approved by AICTE, New Delhi & Affiliated to Osmania University, Hyderabad)
D.No:10TC-111, Kachivanisingaram (V), Uppal, Hyderabad-500088, Telangana.

OBSERVATION:

INPUT		OUTPUT	
6500		6502	
6501		6503	

RESULT:

Thus the 8 bit numbers stored at 6500 & 6501 are subtracted and the result stored at 6502 and 6503.



Exp 3: 8 BIT DATA MULTIPLICATION

AIM:

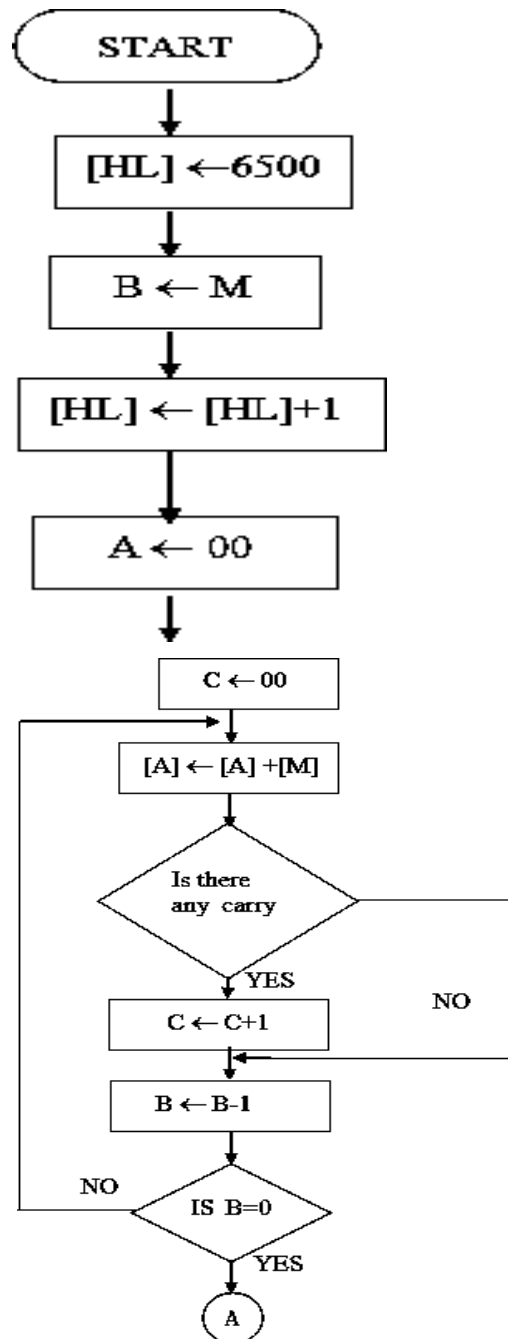
To multiply two 8 bit numbers stored at consecutive memory locations and store the result in memory.

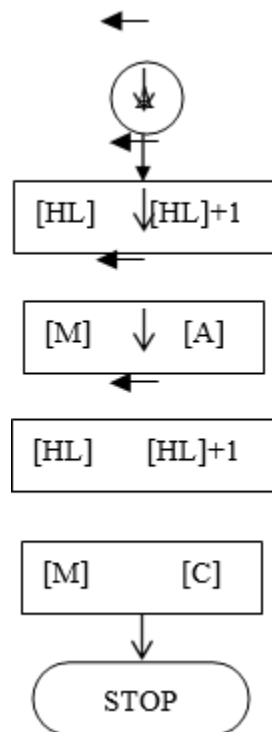
ALGORITHM:

LOGIC: Multiplication can be done by repeated addition.

1. Initialize memory pointer to data location.
2. Move multiplicand to a register.
3. Move the multiplier to another register.
4. Clear the accumulator.
5. Add multiplicand to accumulator
6. Decrement multiplier
7. Repeat step 5 till multiplier comes to zero.
8. The result, which is in the accumulator, is stored in a memory location.

FLOW CHART:





PROGRAM:

ADDRESS	OPCODE	LABEL	MNEMONICS	OPERAND	COMMENT
6100		START	LXI	H, 6500	Initialize HL reg. to 6500
6101					
6102					
6103			MOV	B, M	Transfer first data to reg. B
6104			INX	H	Increment HL reg. to point next mem. Location.
6105			MVI	A, 00H	Clear the acc.
6106					
6107			MVI	C, 00H	Clear C reg for carry
6108					
6109		L1	ADD	M	Add multiplicand multiplier times.

610A		JNC	NEXT	Jump to NEXT if there is no carry
610B				
610C				
610D		INR	C	Increment C reg
610E	NEXT	DCR	B	Decrement B reg
610F		JNZ	L1	Jump to L1 if B is not zero.
6110				
6111				
6112		INX	H	Increment HL reg. to point next mem. Location.
6113		MOV	M, A	Transfer the result from acc. to memory.
6114		INX	H	Increment HL reg. to point next mem. Location.
6115		MOV	M, C	Transfer the result from C reg. to memory.
6116		HLT		Stop the program

OBSERVATION:

INPUT		OUTPUT	
6500		6502	
6501		6503	

RESULT:

Thus the 8-bit multiplication was done in 8085 μ p using repeated addition method.



Exp 4: 8 BIT DATA DIVISION

AIM:

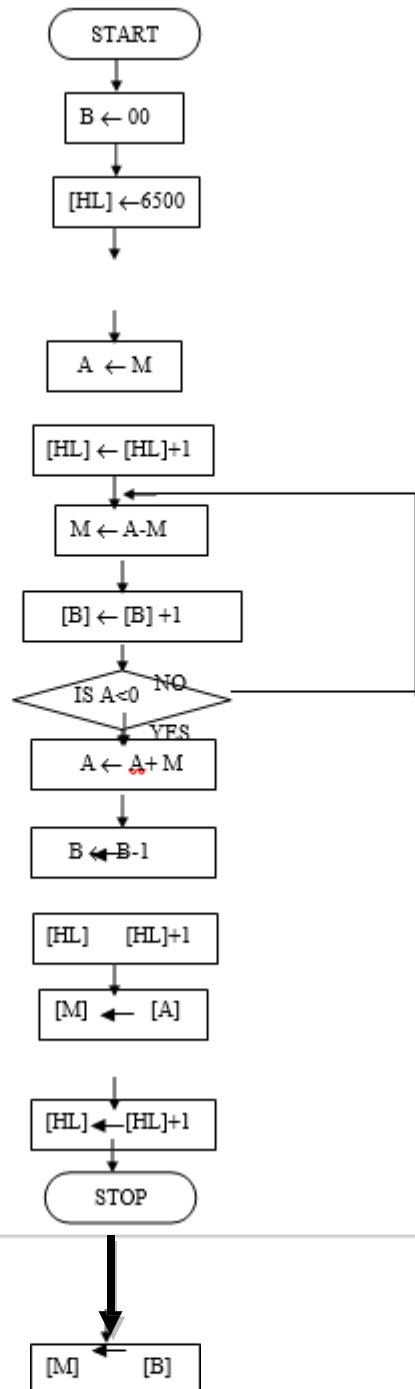
To divide two 8-bit numbers and store the result in memory.

ALGORITHM:

LOGIC: Division is done using the method Repeated subtraction.

1. Load Divisor and Dividend
2. Subtract divisor from dividend
3. Count the number of times of subtraction which equals the quotient
4. Stop subtraction when the dividend is less than the divisor .The dividend now becomes the remainder. Otherwise go to step 2.
5. stop the program execution.

FLOWCHART:



PROGRAM:

ADDRESS	OPCODE	LABEL	MNEMONIC	OPERAND	COMMENTS
6100			MVI	B,00	Clear B reg for quotient
6101					
6102			LXI	H,6500	Initialize HL reg. to
6103					6500H
6104					
6105			MOV	A,M	Transfer dividend to acc.
6106			INX	H	Increment HL reg. to point
					next mem. Location.
6107		LOOP	SUB	M	Subtract divisor from dividend
6108			INR	B	Increment B reg
6109			JNC	LOOP	Jump to LOOP if result does
610A					not yield borrow
610B					
610C			ADD	M	Add divisor to acc.
610D			DCR	B	Decrement B reg
610E			INX	H	Increment HL reg. to point
					next mem. Location.
610F			MOV	M,A	Transfer the remainder from
					acc. to memory.
6110			INX	H	Increment HL reg. to point
					next mem. Location.
6111			MOV	M,B	Transfer the quotient from B
					reg. to memory.
6112			HLT		Stop the program

□



NEIL GOGTE INSTITUTE OF TECHNOLOGY

A unit of Keshav Memorial Technical Educational Society (KMTES)
(Approved by AICTE, New Delhi & Affiliated to Osmania University, Hyderabad)
D.No:10TC-111, Kachivanisingaram (V), Uppal, Hyderabad-500088, Telangana.

OBSERVATION:

S.NO	INPUT		OUTPUT	
	ADDRESS	DATA	ADDRESS	DATA
1	6500		6502	
	6501		6503	
2	6500		6502	
	6501		6503	

RESULT:

Thus an ALP was written for 8-bit division using repeated subtraction method and executed using 8085 μ p kits.

Exp 5: LARGEST ELEMENT IN AN ARRAY

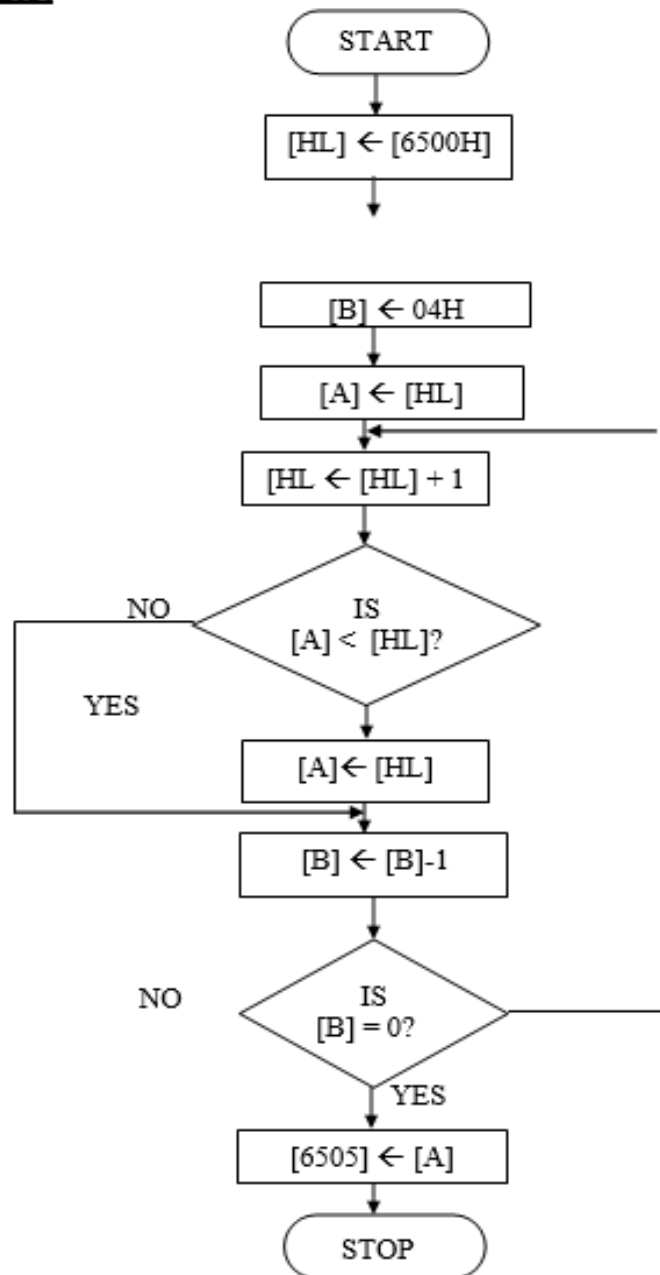
AIM:

To find the largest element in an array.

ALGORITHM:

1. and load it in the accumulator.
2. Initialize a counter (register) with the total number of elements in an array.
3. Decrement the counter by 1.
4. Increment the memory pointer to point to the next element.
5. Compare the accumulator content with the memory content (nextelement).
6. If the accumulator content is smaller, then move the memory content(largest element) to the accumulator. Else continue.
7. Decrement the counter by 1.
8. Repeat steps 5 to 8 until the counter reaches zero
9. Store the result (accumulator content) in the specified memory location. Place all the elements of an array in the consecutive memory locations.
10. Fetch the first element from the memory location

FLOW CHART:



PROGRAM:

ADDRE SS	OPCO DE	LABEL	MNEM ONICS	OPER AND	COMMENTS
6100			LXI	H,6500	Initialize HL reg. to
6101					6500H
6102					
6103			MVI	B,04	Initialize B <u>reg</u> with no. of
6104					comparisons(n-1)
6105			MOV	A,M	Transfer first data to acc.
6106		LOOP1	INX	H	Increment HL reg. to point
					next memory location
6107			CMP	M	Compare M & A
6108			JNC	LOOP	If A is greater than M then go
6109					to loop
610A					
610B			MOV	A,M	Transfer data from M to A <u>reg</u>
610C		LOOP	DCR	B	Decrement B <u>reg</u>
610D			JNZ	LOOP1	If B is not Zero go to loop1
610E					
610F					
6110			STA	6505	Store the result in a memory
6111					location.
6112					
6113			HLT		Stop the program



NEIL GOGTE INSTITUTE OF TECHNOLOGY

A unit of Keshav Memorial Technical Educational Society (KMTES)
(Approved by AICTE, New Delhi & Affiliated to Osmania University, Hyderabad)
D.No:10TC-111, Kachivanisingaram (V), Uppal, Hyderabad-500088, Telangana.

OBSERVATION:

INPUT		OUTPUT	
ADDRESS	DATA	ADDRESS	DATA
6500		6505	
6501			
6502			
6503			
6504			

RESULT:

Thus the largest number in the given array is found out.



NEIL GOGTE INSTITUTE OF TECHNOLOGY

*A unit of Keshav Memorial Technical Educational Society (KMTES)
(Approved by AICTE, New Delhi & Affiliated to Osmania University, Hyderabad)
D.No:10TC-111, Kachivanisingaram (V), Uppal, Hyderabad-500088, Telangana.*

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Exp 6: SMALLEST ELEMENT IN AN ARRAY

AIM:

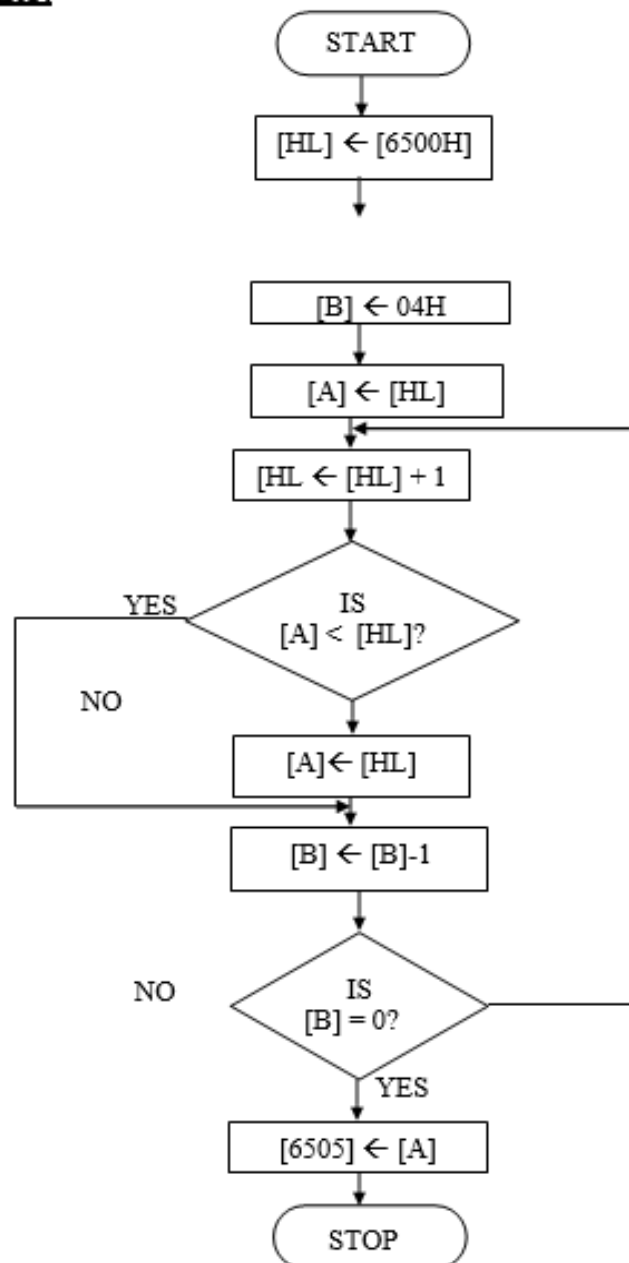
To find the smallest element in an array.

ALGORITHM:

1. Place all the elements of an array in the consecutive memory locations.
2. Fetch the first element from the memory location and load it in the accumulator.
3. Initialize a counter (register) with the total number of elements in an array.
4. Decrement the counter by 1.
5. Increment the memory pointer to point to the next element.
6. Compare the accumulator content with the memory content (nextelement).
7. If the accumulator content is smaller, then move the memory content(largest element) to the accumulator. Else continue.
8. Decrement the counter by 1.
9. Repeat steps 5 to 8 until the counter reaches zero
10. Store the result (accumulator content) in the specified memory location.

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

FLOW CHART:



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

PROGRAM:

ADDRE SS	OPCO DE	LABEL	MNEM ONICS	OPER AND	COMMENTS
6100			LXI	H,6500	Initialize HL reg. to 6500H
6101					
6102					
6103			MVI	B,04	Initialize B reg with no. of comparisons(n-1)
6104					
6105			MOV	A,M	Transfer first data to acc.
6106		LOOP1	INX	H	Increment HL reg. to point next memory location
6107			CMP	M	Compare M & A
6108			JC	LOOP	If A is lesser than M then go to loop
6109					
610A					
610B			MOV	A,M	Transfer data from M to A reg
610C		LOOP	DCR	B	Decrement B reg
610D			JNZ	LOOP1	If B is not Zero go to loop1
610E					
610F					
6110			STA	6505	Store the result in a memory location.
6111					
6112					
6113			HLT		Stop the program

OBSERVATION:

INPUT		OUTPUT	
ADDRESS	DATA	ADDRESS	DATA
6500		6505	
6501			
6502			
6503			
6504			

RESULT:

Thus the smallest number in the given array is found out.



NEIL GOGTE INSTITUTE OF TECHNOLOGY

*A unit of Keshav Memorial Technical Educational Society (KMTES)
(Approved by AICTE, New Delhi & Affiliated to Osmania University, Hyderabad)
D.No:10TC-111, Kachivanisingaram (V), Uppal, Hyderabad-500088, Telangana.*

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Exp 7: SORTING IN ASCENDING ORDER

AIM:

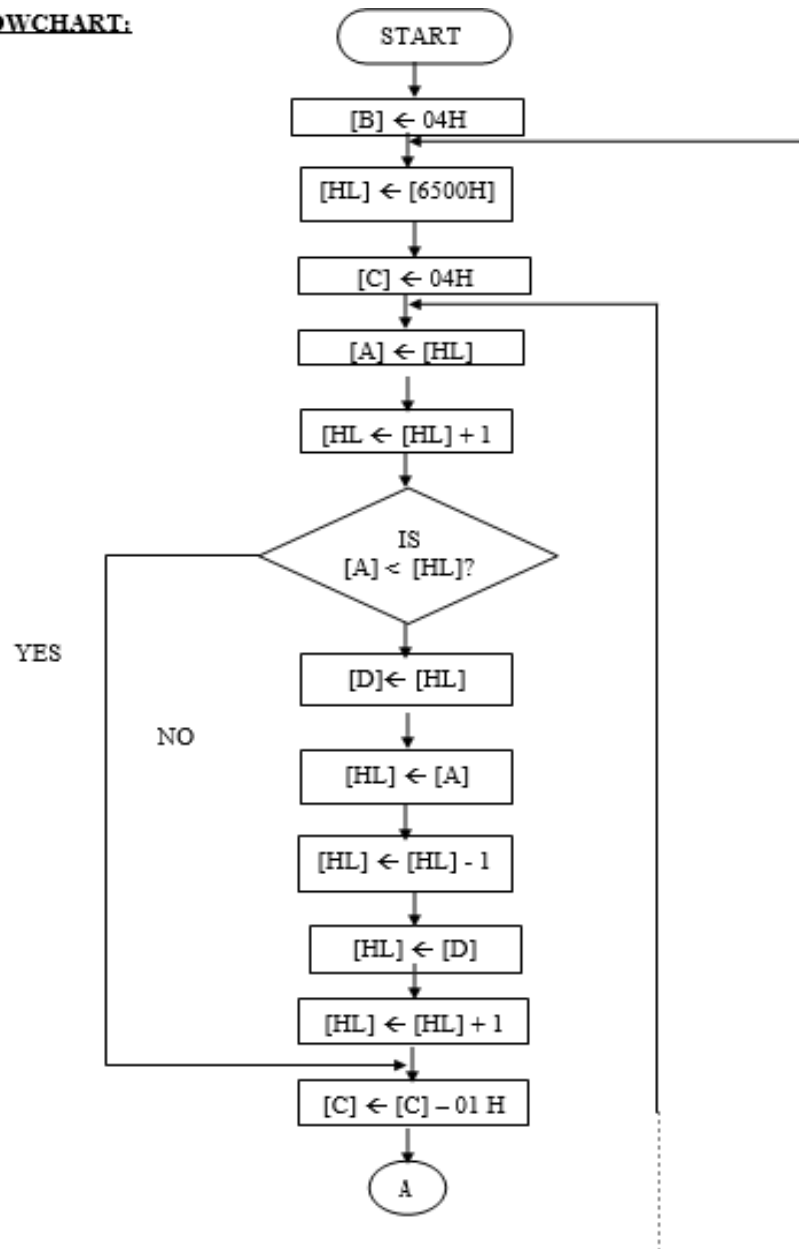
To sort the given number in the ascending order using 8085 microprocessor.

ALGORITHM:

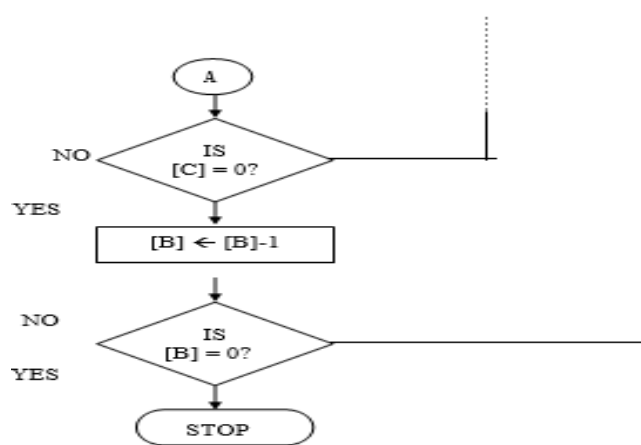
1. Get the numbers to be sorted from the memory locations.
2. Compare the first two numbers and if the first number is larger than second then interchange the number.
3. If the first number is smaller, go to step 4
4. Repeat steps 2 and 3 until the numbers are in required order

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

FLOWCHART:



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING



PROGRAM:

ADDR E	OPCO DE	LABEL	MNEM ONICS	OPER AND	COMMENTS
SS					
6100			MVI	B,04	Initialize B reg with number
6101					of comparisons (n-1)
6102		LOOP 3	LXI	H,6500	Initialize HL reg. to
6103					6500H
6104					
6105			MVI	C,04	Initialize C reg with no. of
6106					comparisons(n-1)
6107		LOOP2	MOV	A,M	Transfer first data to acc.
6108			INX	H	Increment HL reg. to point
					next memory location
6109			CMP	M	Compare M & A
610A			JC	LOOP1	If A is less than M then go to
610B					loop1
610C					
610D			MOV	D,M	Transfer data from M to D reg.
610E			MOV	M,A	Transfer data from acc to M
610F			DCX	H	Decrement HL pair
6110			MOV	M,D	Transfer data from D to M
6111			INX	H	Increment HL pair
6112		LOOP1	DCR	C	Decrement C reg.
6113			JNZ	LOOP2	If C is not zero go to loop2
6114					
6115					
6116			DCR	B	Decrement B reg.
6117			JNZ	LOOP3	If B is not Zero go to loop3
6118					
6119					
611A			HLT		Stop the program



NEIL GOGTE INSTITUTE OF TECHNOLOGY

*A unit of Keshav Memorial Technical Educational Society (KMTES)
(Approved by AICTE, New Delhi & Affiliated to Osmania University, Hyderabad)
D.No:10TC-111, Kachivanisingaram (V), Uppal, Hyderabad-500088, Telangana.*

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

OBSERVATION:

INPUT		OUTPUT	
MEMORY LOCATION	DATA	MEMORY LOCATION	DATA
6500		6500	
6501		6501	
6502		6502	
6503		6503	
6504		6504	

RESULT:

Thus the ascending order program is executed and thus the numbers are arranged in ascending order.



NEIL GOGTE INSTITUTE OF TECHNOLOGY

*A unit of Keshav Memorial Technical Educational Society (KMTES)
(Approved by AICTE, New Delhi & Affiliated to Osmania University, Hyderabad)
D.No:10TC-111, Kachivanisingaram (V), Uppal, Hyderabad-500088, Telangana.*

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Exp 8: SORTING IN DESCENDING ORDER

AIM:

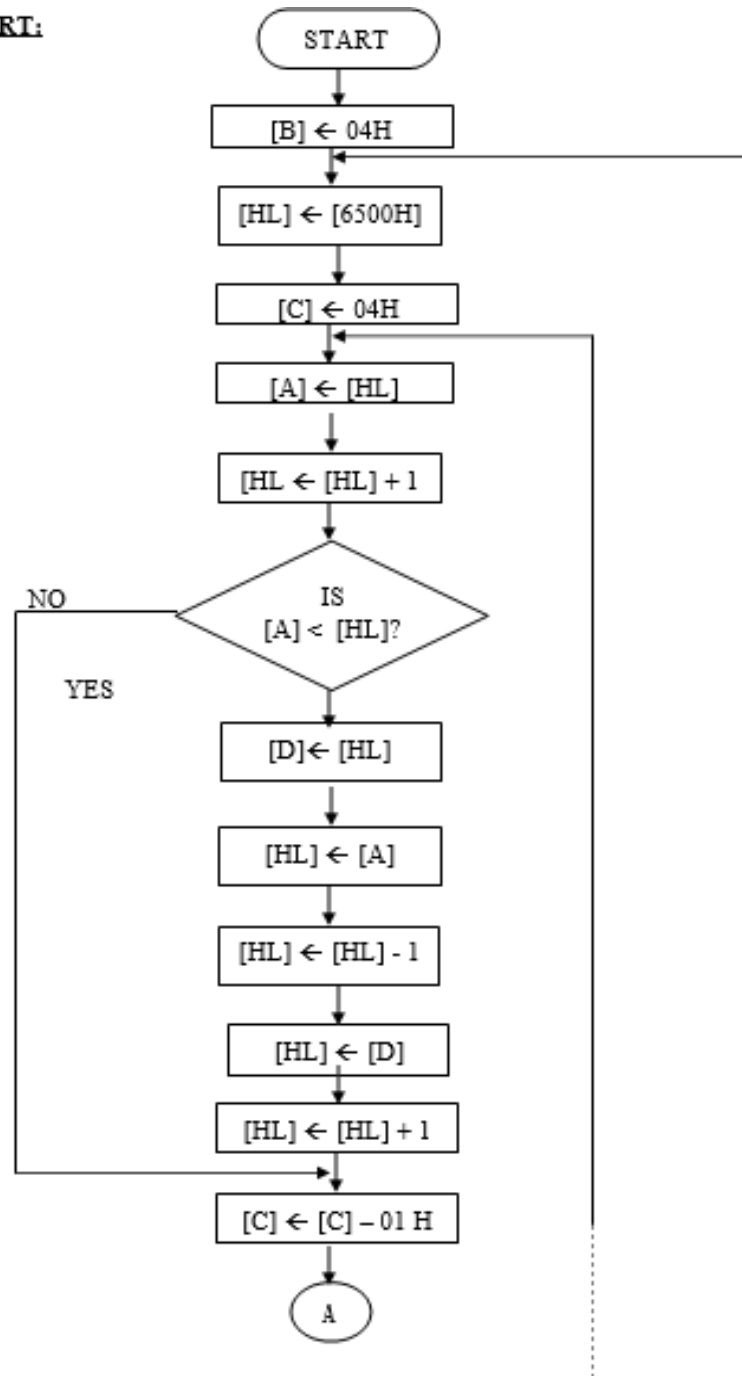
To sort the given number in the descending order using 8085 microprocessors.

ALGORITHM:

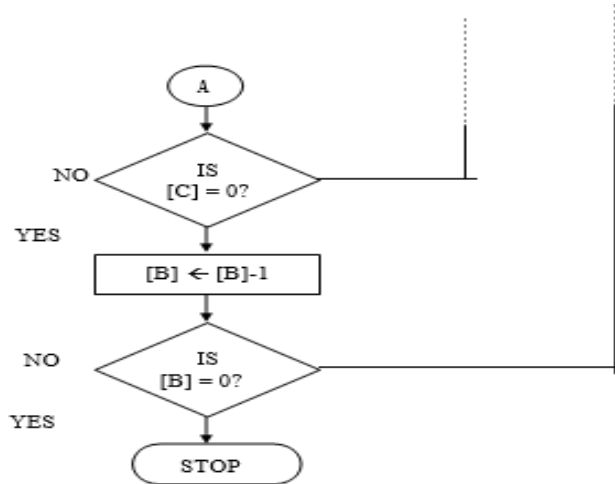
1. Get the numbers to be sorted from the memory locations.
2. Compare the first two numbers and if the first number is smaller than second then
Interchange the number.
3. If the first number is larger, go to step 4
4. Repeat steps 2 and 3 until the numbers are in required order

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

FLOWCHART:



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING



PROGRAM:

ADDRE SS	OPCO DE	LABEL	MNEM ONICS	OPER AND	COMMENTS
6100			MVI	B,04	Initialize B reg with number
6101					of comparisons (n-1)
6102		LOOP 3	LXI	H,6500	Initialize HL reg. to
6103					6500H
6104			MVI	C,04	Initialize C reg with no. of
6105					comparisons(n-1)
6106		LOOP2	MOV	A,M	Transfer first data to acc.
6107			INX	H	Increment HL reg. to point
6108					next memory location
6109			CMP	M	Compare M & A
610A			JNC	LOOP1	If A is greater than M then go
610B					to loop1
610C					
610D			MOV	D,M	Transfer data from M to D reg
610E			MOV	M,A	Transfer data from acc to M
610F			DCX	H	Decrement HL pair
6110			MOV	M,D	Transfer data from D to M
6111			INX	H	Increment HL pair
6112		LOOP1	DCR	C	Decrement C reg
6113			JNZ	LOOP2	If C is not zero go to loop2
6114					
6115					
6116			DCR	B	Decrement B reg
6117			JNZ	LOOP3	If B is not Zero go to loop3
6118					
6119					
611A			HLT		Stop the program



NEIL GOGTE INSTITUTE OF TECHNOLOGY

*A unit of Keshav Memorial Technical Educational Society (KMTES)
(Approved by AICTE, New Delhi & Affiliated to Osmania University, Hyderabad)
D.No:10TC-111, Kachivanisingaram (V), Uppal, Hyderabad-500088, Telangana.*

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

OBSERVATION:

INPUT		OUTPUT	
MEMORY LOCATION	DATA	MEMORY LOCATION	DATA
6500		6500	
6501		6501	
6502		6502	
6503		6503	
6504		6504	

RESULT:

Thus the descending order program is executed and thus the numbers are arranged in descending order.



NEIL GOGTE INSTITUTE OF TECHNOLOGY

*A unit of Keshav Memorial Technical Educational Society (KMTES)
(Approved by AICTE, New Delhi & Affiliated to Osmania University, Hyderabad)
D.No:10TC-111, Kachivanisingaram (V), Uppal, Hyderabad-500088, Telangana.*

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Exp 09: CODE CONVERSION –DECIMAL TO HEXDECIMAL

AIM:

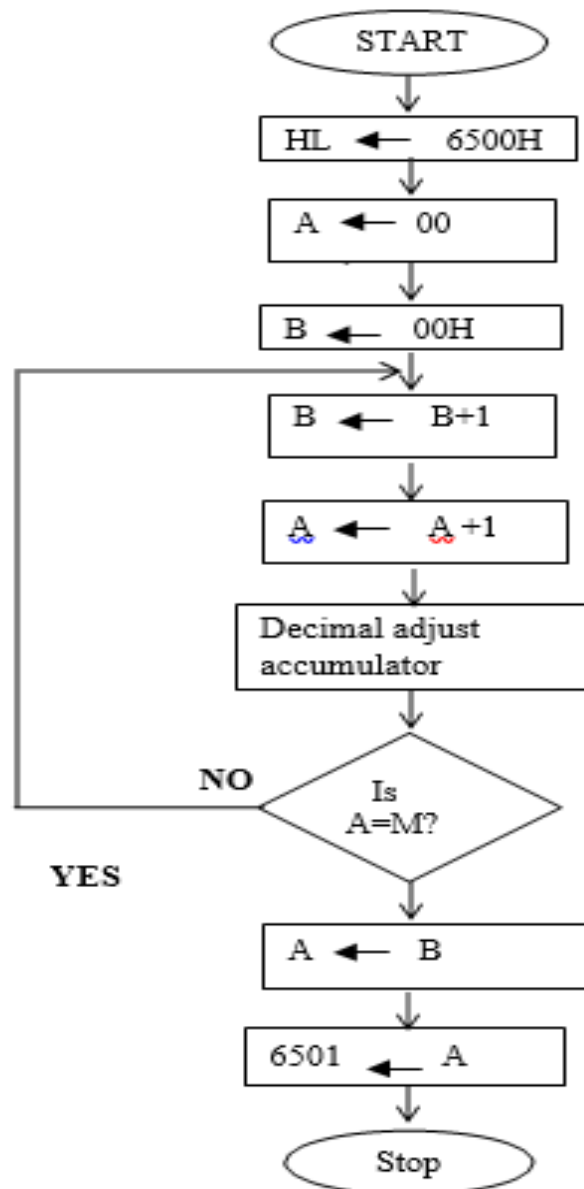
To convert a given decimal number to hexadecimal.

ALGORITHM:

1. Initialize the memory location to the data pointer.
2. Increment B register.
3. Increment accumulator by 1 and adjust it to decimal every time.
4. Compare the given decimal number with accumulator value.
5. When both matches, the equivalent hexadecimal value is in B register.
6. Store the resultant in memory location.

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

FLOWCHART:



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

PROGRAM:

ADDRE SS	OPCO DE	LABEL	MNEM ONICS	OPER AND	COMMENTS
6100			LXI	H,6500	Initialize HL reg. to 6500H
6101					
6102					
6103			MVI	A,00	Initialize A register.
6104					
6105			MVI	B,00	Initialize B register..
6106					
6107		LOOP	INR	B	Increment B reg.
6108			ADI	01	Increment A reg
6109					
610A			DAA		Decimal Adjust Accumulator
610B			CMP	M	Compare M & A
610C			JNZ	LOOP	If acc and given number are not equal, then go to LOOP
610D					
610E					
610F			MOV	A,B	Transfer B reg to acc.
6110			STA	6501	Store the result in a memory location.
6111					
6112					
6113			HLT		Stop the program

RESULT:

INPUT		OUTPUT	
ADDRESS	DATA	ADDRESS	DATA
6500		6501	

Thus an ALP program for conversion of decimal to hexadecimal was written and executed.



NEIL GOGTE INSTITUTE OF TECHNOLOGY

*A unit of Keshav Memorial Technical Educational Society (KMTES)
(Approved by AICTE, New Delhi & Affiliated to Osmania University, Hyderabad)
D.No:10TC-111, Kachivanisingaram (V), Uppal, Hyderabad-500088, Telangana.*

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Exp 10: CODE CONVERSION –HEXADECIMAL TO DECIMAL

AIM:

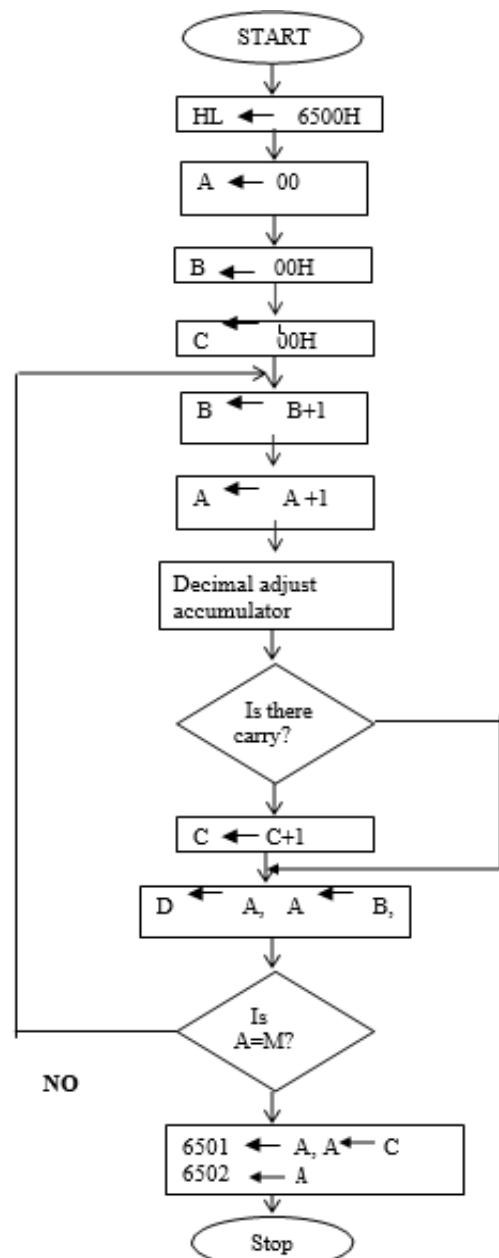
To convert a given hexadecimal number to decimal.

ALGORITHM:

1. Initialize the memory location to the data pointer.
2. Increment B register.
3. Increment accumulator by 1 and adjust it to decimal every time.
4. Compare the given hexadecimal number with B register value.
5. When both match, the equivalent decimal value is in A register.
6. Store the resultant in memory location.

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

FLOWCHART:



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

PROGRAM:

ADDRE SS	OPCO DE	LABEL	MNEM ONICS	OPER AND	COMMENTS
6100			LXI	H,6500	Initialize HL reg. to 6500H
6101					
6102			MVI	A,00	Initialize A register.
6103					
6104			MVI	B,00	Initialize B register.
6105					
6106			MVI	C,00	Initialize C register for carry.
6107					
6108					
6109		LOOP	INR	B	Increment B reg.
610A			ADI	01	Increment A reg.
610B					
610C			DAA		Decimal Adjust Accumulator
610D			JNC	NEXT	If there is no carry go to NEXT.
610E					
610F					
6110			INR	C	Increment c register.
6111		NEXT	MOV	D,A	Transfer A to D
6112			MOV	A,B	Transfer B to A
6113			CMP	M	Compare M & A
6114			MOV	A,D	Transfer D to A
6115			JNZ	LOOP	If acc, and given number are not equal, then go to LOOP
6116					
6117					
6118			STA	6501	Store the result in a memory location.
6119					
611A					
611B			MOV	A,C	Transfer C to A
611C			STA	6502	Store the carry in another memory location.
611D					
611E					
611F			HLT		Stop the program

RESULT:

INPUT		OUTPUT	
ADDRESS	DATA	ADDRESS	DATA
6500		6501	
		6502	

Thus an ALP program for conversion of hexadecimal to decimal was written and executed.



NEIL GOGTE INSTITUTE OF TECHNOLOGY

*A unit of Keshav Memorial Technical Educational Society (KMTES)
(Approved by AICTE, New Delhi & Affiliated to Osmania University, Hyderabad)
D.No:10TC-111, Kachivanisingaram (V), Uppal, Hyderabad-500088, Telangana.*

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Exp 11:BCD ADDITION

AIM:

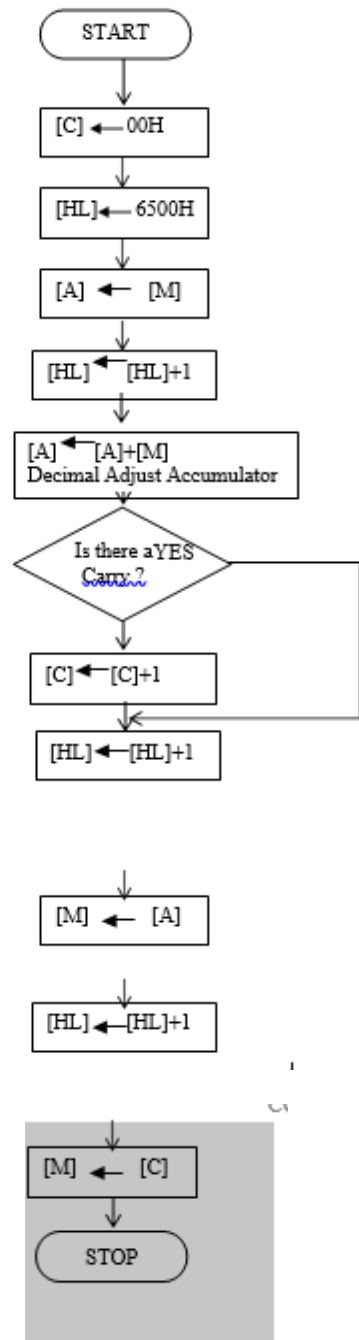
To add two 8 bit BCD numbers stored at consecutive memory locations.

ALGORITHM:

1. Initialize memory pointer to data location.
2. Get the first number from memory in accumulator.
3. Get the second number and add it to the accumulator
4. Adjust the accumulator value to the proper BCD value using DAA instruction.
5. Store the answer at another memory location.

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

FLOW CHART:



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

PROGRAM:

ADDRESS	OPCODE	LABEL	MNEMONICS	OPERAND	COMMENT
6100		START	MVI	C, 00	Clear C reg.
6101					
6102			LXI	H, 6500	Initialize HL reg. to 6500H
6103					
6104					
6105			MOV	A, M	Transfer first data to accumulator
6106			INX	H	Increment HL reg. to point next memory Location.
6107			ADD	M	Add first number to acc. Content.
6108			DAA		Decimal adjust accumulator
6109			JNC	L1	Jump to location if result does not yield carry.
610A					
610B					
610C			INR	C	Increment C reg.
610D		L1	INX	H	Increment HL reg. to point next memory Location.
610E			MOV	M, A	Transfer the result from acc. to memory.
610F			INX	H	Increment HL reg. to point next memory Location.
6110			MOV	M, C	Move carry to memory
6111			HLT		Stop the program

□

OBSERVATION:

INPUT		OUTPUT	
6500		6502	
6501		6503	

RESULT:

Thus the 8 bit BCD numbers stored at 6500 & 6501 are added and the result stored at 6502 & 6503.



NEIL GOGTE INSTITUTE OF TECHNOLOGY

*A unit of Keshav Memorial Technical Educational Society (KMTES)
(Approved by AICTE, New Delhi & Affiliated to Osmania University, Hyderabad)
D.No:10TC-111, Kachivanisingaram (V), Uppal, Hyderabad-500088, Telangana.*

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Exp 12: BCD SUBTRACTION

AIM:

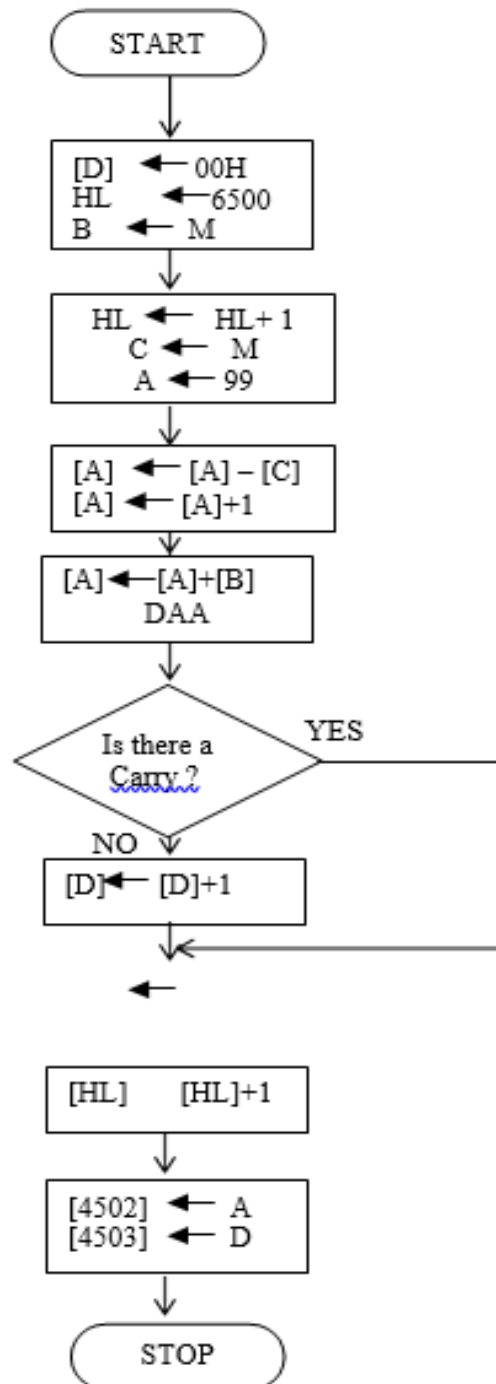
To Subtract two 8 bit BCD numbers stored at consecutive memory locations.

ALGORITHM:

1. Load the minuend and subtrahend in two registers.
2. Initialize Borrow register to 0.
3. Take the 100's complement of the subtrahend.
4. Add the result with the minuend which yields the result.
5. Adjust the accumulator value to the proper BCD value using DAA instruction. If there is a carry ignore it.
6. If there is no carry, increment the carry register by 1
7. Store the content of the accumulator (result) and borrow register in the specified memory location

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

FLOW CHART:



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

PROGRAM:

ADDRESS	OPCODE	LABEL	MNEMONICS	OPERAND	COMMENT
6100		START	MVI	D, 00	Clear D reg.
6101					
6102			LXI	H, 6500	Initialize HL reg. to 6500H
6103					
6104					
6105			MOV	B, M	Transfer first data to accumulator
6106			INX	H	Increment HL reg. to point next mem. Location.
6107			MOV	C, M	Move second no. to B reg.
6108			MVI	A, 99	Move 99 to the Accumulator
6109					
610A			SUB	C	Subtract [C] from acc. Content.
610B			INR	A	Increment A register
610C			ADD	B	Add [B] with [A]
610D			DAA		Adjust Accumulator value for Decimal digits
610E			JC	LOOP	Jump on carry to loop
610F					
6110					
6111			INR	D	Increment D reg.
6112		LOOP	INX	H	Increment HL register pair
6113			MOV	M, A	Move the Acc content to the memory location
6114			INX	H	Increment HL reg. to point next mem. Location.
6115			MOV	M, D	Transfer D register content to memory.
6116			HLT		Stop the program

OBSERVATION:

INPUT		OUTPUT	
6500		6502	
6501		6503	

RESULT:

Thus the 8 bit BCD numbers stored at 6500 & 6501 are subtracted and the result stored at 6502 & 6503.

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Exp 13: INTRODUCTION TO 8255 PPI

AIM:

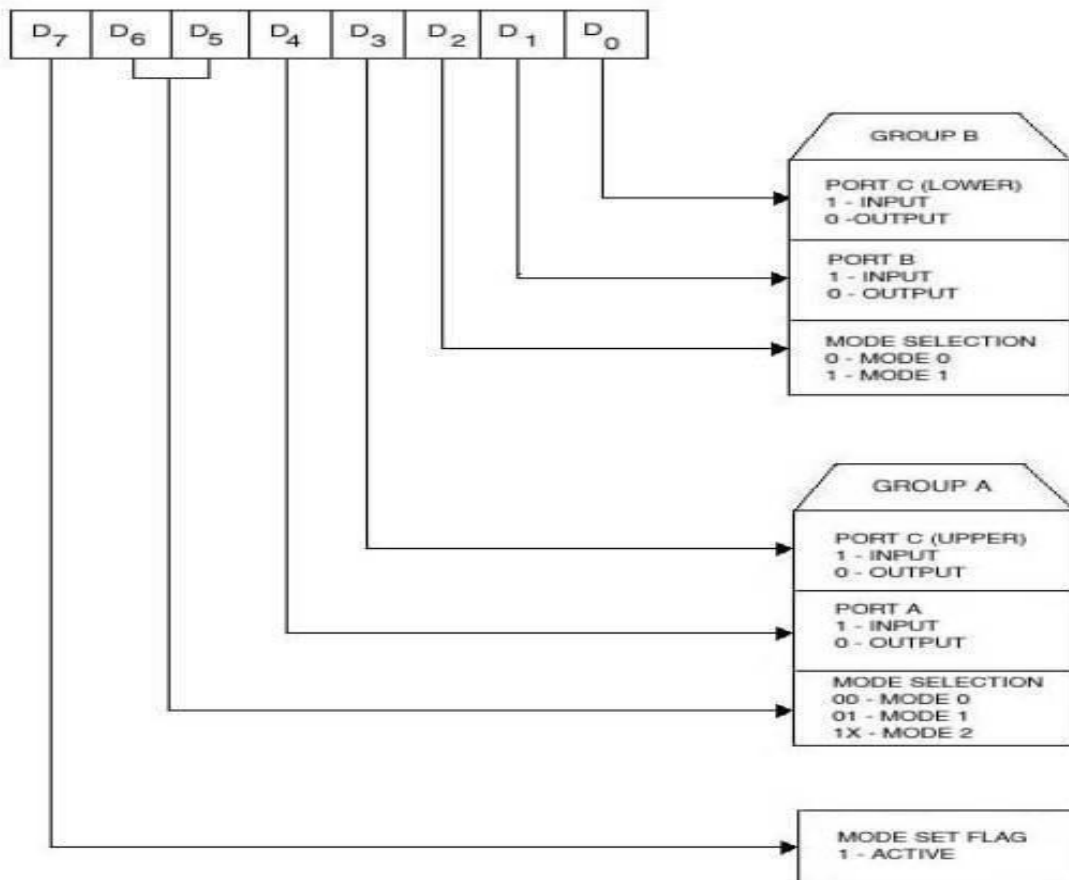
To interface programmable peripheral interface 8255 with 8085 and study its characteristics in mode0, mode1 and BSR mode.

APPARATUS REQUIRED:

8085 μ p kit, 8255 Interface board, DC regulated power supply, VXT parallel bus

I/O MODES:

Control Word:



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

MODE 0 – SIMPLE I/O MODE:

This mode provides simple I/O operations for each of the three ports and is suitable for synchronous data transfer. In this mode all the ports can be configured either as input or output port.

Let us initialize port A as input port and port B as output port

PROGRAM:

ADDRESS	OPCODES	LABEL	MNEMONICS	OPERAND	COMMENTS
6100		START:	MVI	A, 90	Initialize port A as Input
6101					and Port B as output.
6102			OUT	C6	Send Mode Control
6103					Word
6104			IN	C0	Read from Port A
6105					
6106			OUT	C2	Display the data in port
6107					B
6108			STA	6200	Store the data read from
6109					Port A in 6200
610A					
610B			HLT		Stop the program.

MODE1 STROBED I/O MODE:

In this mode, port A and port B are used as data ports and port C is used as control signals for strobed I/O data transfer.

Let us initialize port A as input port in mode1

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

MAIN PROGRAM:

ADDRESS	OPCODES	LABEL	MNEMONICS	OPERAND	COMMENTS
6100		START:	MVI	A, B4	Initialize port A as Input
6101					port in mode 1.
6102			OUT	C6	Send Mode Control
6103					Word
6104			MVI	A,09	Set the PC4 bit for INTE
					A
6105					
6106			OUT	C6	Display the data in port
					B
6107					
			EI		
6108			MVI	A,08	Enable RST5.5
6109					
610A			SIM		
			EI		
610B			HLT		Stop the program.

ISR (Interrupt Service Routine)

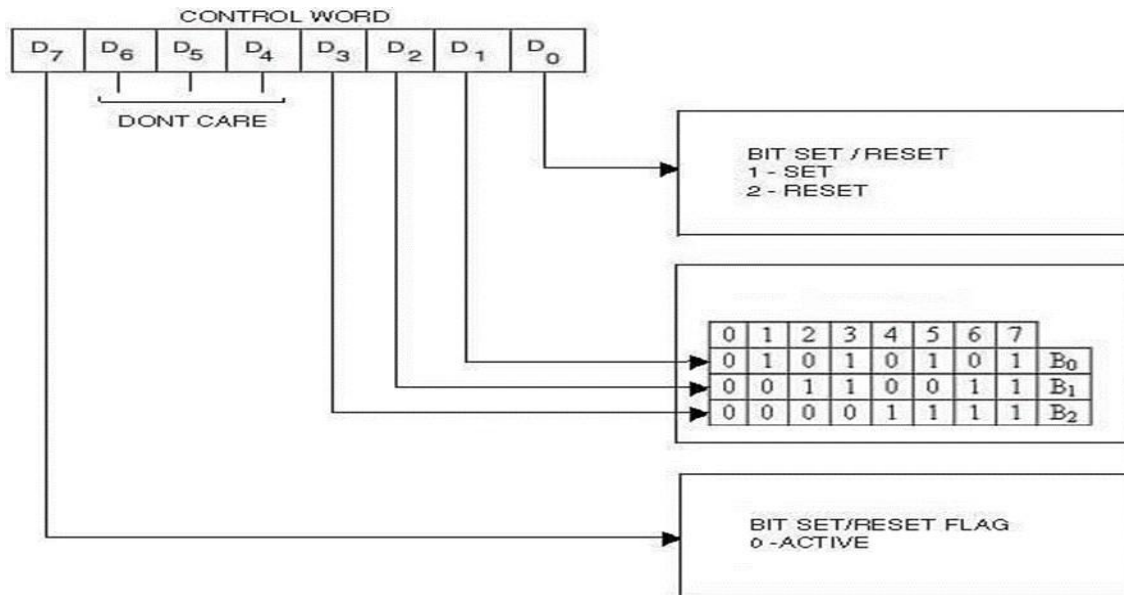
ADDRESS	OPCODES	LABEL	MNEMONICS	OPERAND	COMMENTS
6200		START:	IN	C0	Read from port A
6201					
6202			STA	6500	Store in 6500.
6203					
6204					
6205			HLT		Stop the program.

Sub program:

ADDRESS	OPCODES	LABEL	MNEMONICS	OPERAND	COMMENTS
605E			JMP	6200	Go to 6200
605F					
6060					

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

BSR MODE (Bit Set Reset mode)



Any lines of port c can be set or reset individually without affecting other lines using this mode. Let us set PC0 and PC3 bits using this mode.

PROGRAM:

ADDRESS	OPCODES	LABEL	MNEMONICS	OPERAND	COMMENTS
6100		START:	MVI	A, 01	Set PC0
6101					
6102			OUT	C6	Send Mode Control
6103					word
6104			MVI	A, 07	Set PC3
6105					
6106			OUT	C6	Send Mode Control
6107					word
6109			HLT		Stop the program.

RESULT:

Thus 8255 is interfaced and its characteristics in mode 0, mode 1 and BSR mode is studied.



NEIL GOGTE INSTITUTE OF TECHNOLOGY

*A unit of Keshav Memorial Technical Educational Society (KMTES)
(Approved by AICTE, New Delhi & Affiliated to Osmania University, Hyderabad)
D.No:10TC-111, Kachivanisingaram (V), Uppal, Hyderabad-500088, Telangana.*

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Exp 14: INTERFACING STEPPER MOTOR

Aim: To rotate stepper motor in forward and reverse direction.

Apparatus required:

Stepper motor, 8085 microprocessor kit, (0-5V) power supply.

Algorithm:

Step 1: Load the 'HL' pair with value from table

Step 2: Move it to 'B' register for setting the counter

Step 3: Move the memory value to accumulator and display it by control word

Step 4: Load 'DE' register pair with FFFF for starting delay subroutine

Step 5: Run the delay loop control D-register becomes

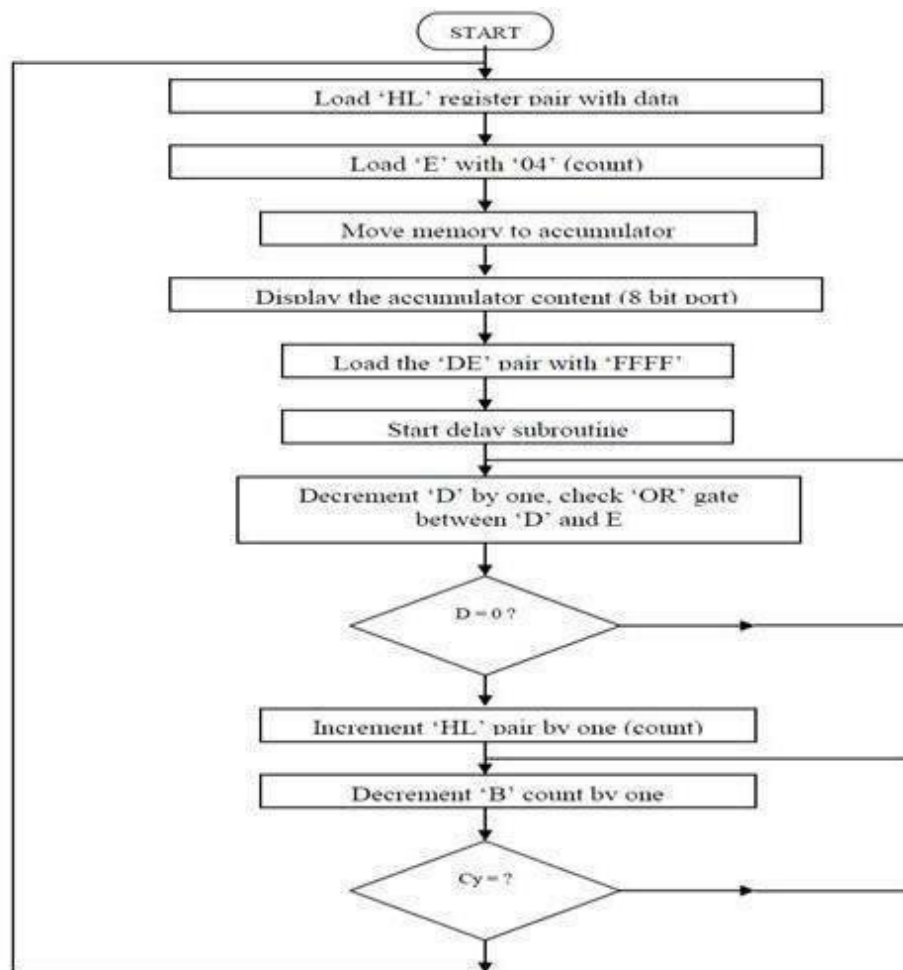
zero. Step 6: Increment 'H' address for next value from

table Step 7: Jump on no zero

Step 8: When B = 0, go to start and restart the program

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

FLOWCHART:



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

STEPPER MOTOR (CLOCKWISE)



6000,6001	3E,80		MVI	A,08
6002,6003	D3,0B		OUT	0B
6004,6005	3E,00		MVI	A,00
6006,6007	D3,08		OUT	08
6008,6009	3E,08	START	MVI	A,01
600A,600B	D3,08		OUT	08
600C,600D,600E	CD,27,60		CALL	DELAY
600F,6010	3E,04		MVI	A,02
6011,6012	D3,08		OUT	08
6013,6014,6015	CD,27,60		CALL	DELAY 1
6016,6017	3E,02		MVI	A,04
6018,6019	D3,08		OUT	08
610A,610B,610C	CD,27,60		CALL	DELAY 1
610D,610E	3E,01		MVI	A,08
601F,6020	D3,08		OUT	08
6021,6022,6023	CD,27,60		CALL	DELAY 1
6024,6025,6026	C3,08,60		JMP	START
6027,6028,6029	11,FF,FF	DELAY	LXI	D,FFFF
602A,602B,602C	CD,15,06		CALL	DELAY 0FFF
602D,602E,602F	CD,15,06		CALL	DELAY 0615
6030,6031,6032	CD,15,06		CALL	DELAY 0615
6033	C9		RET	



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

STEPPER MOTOR (ANTI-CLOCKWISE)

6000,6001	3E,80		MVI	A,08
6002,6003	D3,0B		OUT	0B
6004,6005	3E,00		MVI	A,00
6006,6007	D3,08		OUT	08
6008,6009	3E,08	START	MVI	A,08
600A,600B	D3,08		OUT	08
600C,600D,600E	CD,27 60		CALL	DELAY
600F,6010	3E,04		MVI	A,04
6011,6012	D3,08		OUT	08
6013,6014,6015	CD,27,60		CALL	DELAY 1
6016,6017	3E,02		MVI	A,02
6018,6019	D3,08		OUT	08
610A,610B,610C	CD,27,60		CALL	DELAY 1
610D,610E	3E,01		MVI	A,01
601F,6020	D3,08		OUT	08
6021,6022,6023	CD,27,60		CALL	DELAY 1
6024,6025,6026	C3,08,60		JMP	START
6027,6028,6029	11,FF,FF	DELAY	LXI	D,FFFF
602A,602B,602C	CD,15,06		CALL	DELAY 0FFF
602D,602E,602F	CD,15,06		CALL	DELAY 0615
6030,6031,6032	CD,15,06		CALL	DELAY 0615
6033	C9		RET	

RESULT: Inter facing of stepper motor in forward and reverse direction are studied.

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Exp 15: INTERFACING OF DAC

Aim: To generate different waveforms through DAC using 8085 and 8255.

Apparatus required: 8085 Microprocessor trainer kit, DAC interface board, Regulated Power supply, CRO.

Description: Seven segment display interface is connected over J2 of the trainer. When the trainer kit in KEYBOARD or SERIAL mode the square, triangular, sawtooth and staircase can be displayed on CRO.

DAC SQUARE WAVE

ADDRESS	OPCODE	LABEL	MNEMONICS	OPERAND
6100,6101	3E,80		MVI	A,80
6102,6103	D3,0B		OUT	0B
6104,6105	3E,00	UP	MVI	A,00
6106,6107	D3,08		OUT	08
6108,6109	06,D7		MVI	B,D7
610A	05	XYZ	DCR	B
610B,610C,610D	C2,0A,61		JNZ	XYZ
610E,610F	3E,FF		MVI	A,FF
6110,6111	D3,08		OUT	08
6112,6113	06,D7		MVI	B,D7
6114	05	ABC	DCR	B
6115,6116,6117	C2,14,61		JNZ	ABC
6118,6119,611A	C3,04,61		JMP	UP

RESULT: Inter facing of different waveforms through DAC using 8085 and 8255

are studied.



NEIL GOGTE INSTITUTE OF TECHNOLOGY

*A unit of Keshav Memorial Technical Educational Society (KMTES)
(Approved by AICTE, New Delhi & Affiliated to Osmania University, Hyderabad)
D.No:10TC-111, Kachivanisingaram (V), Uppal, Hyderabad-500088, Telangana.*

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Exp 16: INTERFACING OF TRAFFIC LIGHT CONTROLLER

Aim: To control the traffic light system using 8085 and 8255.

Apparatus required:

8085 Microprocessor trainer kit, Traffic light interface board, Regulated Power supply.

Description: The traffic system moves from one state to next state. By changing the delay between two signals one can change the speed of traffic.

8255 port addresses:

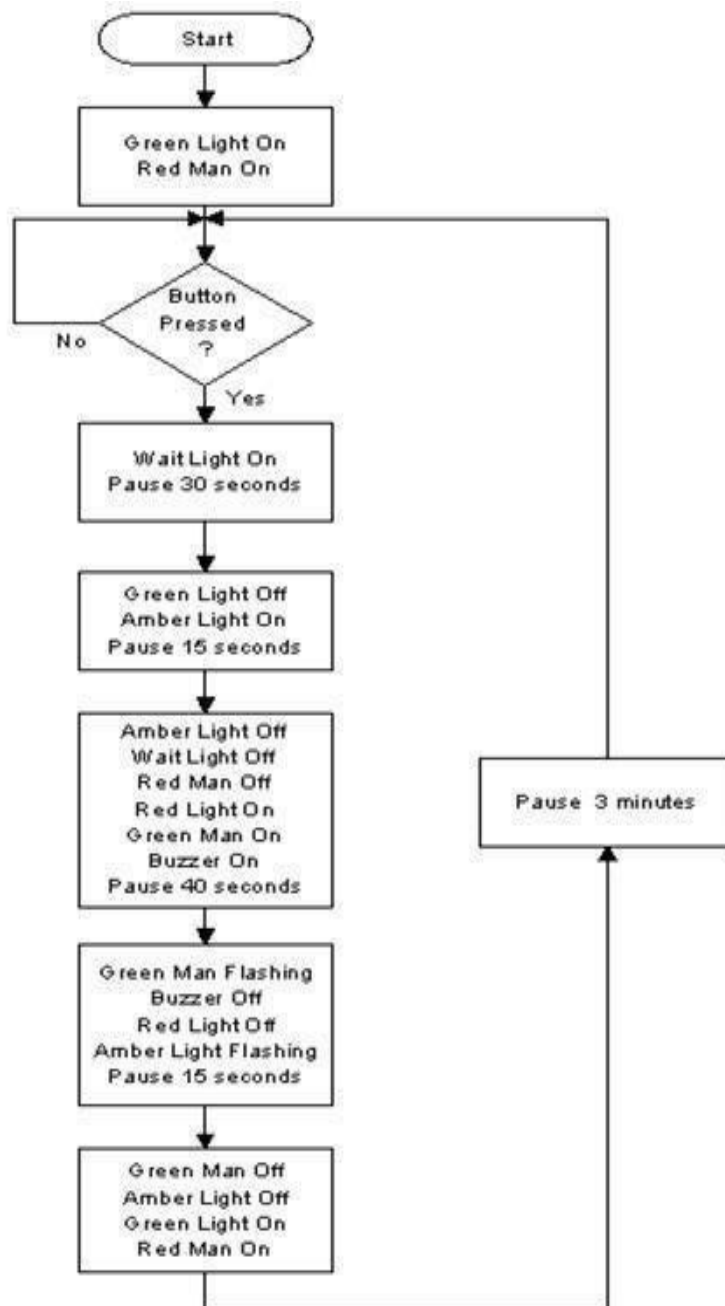
PORT A - 40H

PORT B - 41H

PORT C - 42H

Control word register -43H

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING



RESULT: The traffic light controller system using 8085 and 8255 are studied

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

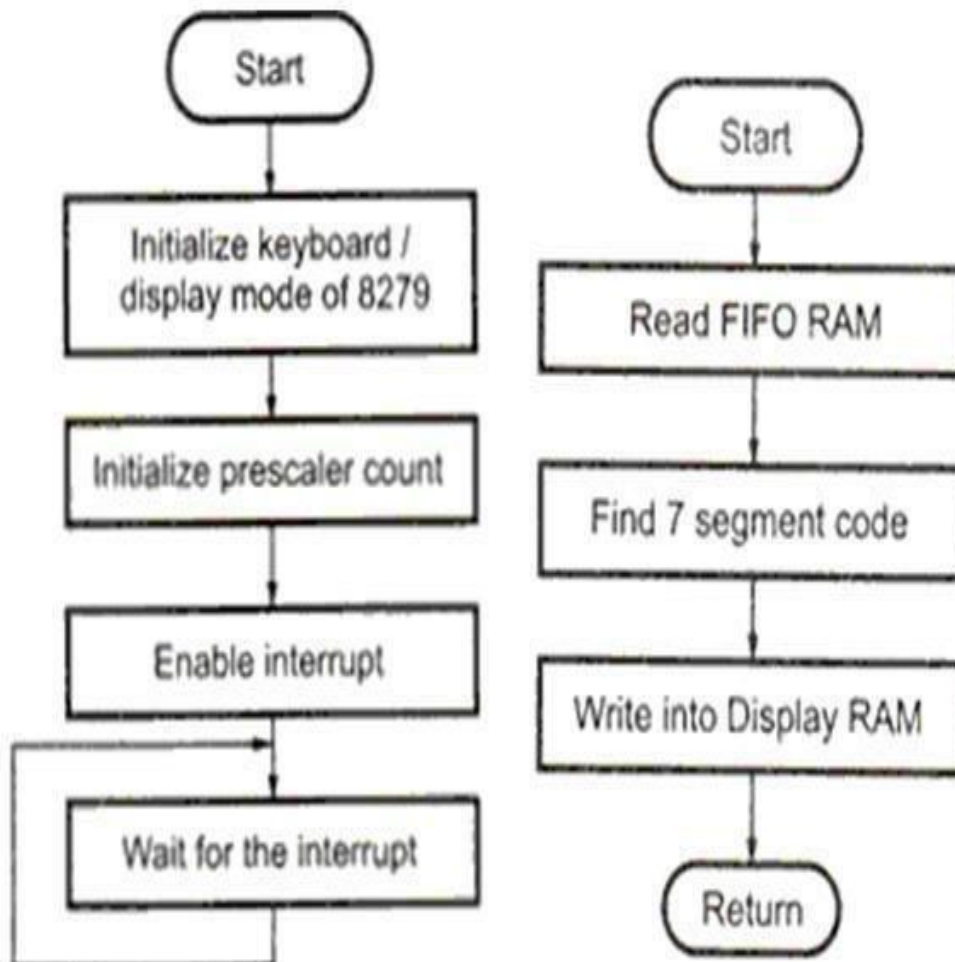
Exp 17: INTERFACING OF 8279

Aim: To interface 8279 to 8085 microprocessor.

Apparatus required:

8085 Microprocessor trainer kit, 8279 keyboard/display interface board, Regulated Powersupply.

Description: 8279 keyboard/display interface is connected over J2 of the trainer. When the trainer kit in KEYBOARD or SERIAL mode the pressed key can be displayed on display unit.



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Exp 18:DISPLAY INTERFACE

Aim: To display LED'S using 8085 and 8255.

Apparatus required: 8085 Microprocessor trainer kit, LED interface board, Regulated Powersupply.

Description: LED interface is connected over J2 of the trainer. When the trainer kit in KEYBOARD or SERIAL mode it scans system key codes.

PROGRAM:

ADDRESS	OPCODE	LABEL	MNEMONICS	OPERAND
6200,6201	3E,80		MVI	A,80
6202,6203	D3,0B		OUT	0B
6204,6205	3E,00	UP	MVI	A,00
6206,6207	D3,0A		OUT	0A
6208,6209,620A	CD,CE,7D		CALL	DELAY
620B,620C	3E,01		MVI	A,01
620D,620E	D3,0A		OUT	0A
620F,6210,6211	CD,15,62		CALL	DELAY
6212,6213,6214	C3,04,62		JMP	UP
6215,6216,6217	11,FF,FF	DELAY	LXI	D,FFFF
6218,6219,621A	CD,FF,0F		CALL	DELAY (0FFF)
621B,621C,621D	CD,FF,0F		CALL	DELAY (0FFF)
621E,621F,6220	CD,FF,0F		CALL	DELAY (0FFF)
6221	C9		RET	

RESULT:

The display LED'S using 8085 and 8255 experiment studied.